

## 32 of 689

**binary digit** Also called a bit. One of the two numbers in base 2 (0 or 1) that are the components of information.

can execute. Given the sophistication of modern programming languages and the simple instructions executed by the hardware, the translation from a high-level language program to hardware instructions is complex. We will give a brief overview of the process and return to the subject in Chapter 2.

### From a High-Level Language to the Language of Hardware

To actually speak to an electronic machine, you need to send electrical signals. The easiest signals for machines to understand are *on* and *off*, and so the machine alphabet is just two letters. Just as the 26 letters of the English alphabet do not limit how much can be written, the two letters of the computer alphabet do not limit what computers can do. The two symbols for these two letters are the numbers 0 and 1, and we commonly think of the machine language as numbers in base 2, or *binary numbers*. We refer to each "letter" as a **binary digit** or **bit**. Computers are slaves to our commands, which are called instructions. Instructions, which are just collections of bits that the computer understands, can be thought of as numbers. For example, the bits

```
1000110010100000
```

tell one computer to add two numbers. Chapter 3 explains why we use numbers for instructions *and* data; we don't want to steal that chapter's thunder, but using numbers for both instructions and data is a foundation of computing.

## 1.2 Below Your Program

13

The first programmers communicated to computers in binary numbers, but this was so tedious that they quickly invented new notations that were closer to the way humans think. At first these notations were translated to binary by hand, but this process was still tiresome. Using the machine to help program the machine, the pioneers invented programs to translate from symbolic notation to binary. The first of these programs was named an **assembler**. This program translates a symbolic version of an instruction into the binary version. For example, the programmer would write

```
add A,B
```

and the assembler would translate this notation into

```
1000110010100000
```

This instruction tells the computer to add the two numbers A and B. The name coined for this symbolic language, still used today, is **assembly language**.

Although a tremendous improvement, assembly language is still far from the notation a scientist might like to use to simulate fluid flow or that an accountant might use to balance the books. Assembly language requires the programmer to write one line for every instruction that the machine will follow, forcing the programmer to think like the machine.

The recognition that a program could be written to translate a more powerful language into computer instructions was one of the great breakthroughs in the early days of computing. Programmers today owe their productivity—and their sanity—to the creation of **high-level programming languages** and compilers that translate programs in such languages into instructions.

A compiler enables a programmer to write this high-level language expression:

```
A + B
```

The compiler would compile it into this assembly language statement:

```
add A,B
```

The assembler would translate this statement into the binary instruction that tells the computer to add the two numbers A and B:

```
1000110010100000
```

Figure 1.4 shows the relationships among these programs and languages.

High-level programming languages offer several important benefits. First, they allow the programmer to think in a more natural language, using English words and algebraic notation, resulting in programs that look much more like text than like tables of cryptic symbols (see Figure 1.4). Moreover, they allow languages to

**assembler** A program that translates a symbolic version of instructions into the binary version.

**assembly language** A symbolic representation of machine instructions.

**high-level programming language** A portable language such as C, Fortran, or Java composed of words and algebraic notation that can be translated by a compiler into assembly language.

1000110010100000

33 of 689 onships among these programs and languages. High-level languages offer several important benefits. First, they allow the programmer to think in a more natural language, using English words and algebraic notation, resulting in programs that look much more like text than like tables of cryptic symbols (see Figure 1.4). Moreover, they allow languages to

## Chapter 1 Computer Abstractions and Technology

High-level  
language  
program  
(in C)

```
swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Compiler

Assembly  
language  
program  
(for MIPS)

```
swap:
    muli $2, $5, 4
    add $2, $4, $2
    lw $15, 0($2)
    lw $16, 4($2)
    sw $16, 0($2)
    sw $15, 4($2)
    jr $31
```

Assembler

Binary machine  
language  
program  
(for MIPS)

```
000000001010000100000000000011000
000000000000110000001100000100001
100011000110001000000000000000000
100011001111001000000000000001000
101011001111001000000000000000000
101011000110001000000000000001000
0000001111100000000000000001000
```

**FIGURE 1.4 C program compiled into assembly language and then assembled into binary machine language.** Although the translation from high-level language to binary machine language is shown in two steps, some compilers cut out the middleman and produce binary machine language directly. These languages and this program are examined in more detail in Chapter 2.

be designed according to their intended use. Hence, Fortran was designed for scientific computation, Cobol for business data processing, Lisp for symbol manipulation, and so on.

The second advantage of programming languages is improved programmer productivity. One of the few areas of widespread agreement in software development is that it takes less time to develop programs when they are written in languages that require fewer lines to express an idea. Conciseness is a clear advantage of high-level languages over assembly language.

## 1.3 Under the Covers

The final advantage is that programming languages allow programs to be independent of the computer on which they were developed, since compilers and assemblers can translate high-level language programs to the binary instructions of any machine. These three advantages are so strong that today little programming is done in assembly language.

## 1.3 Under the Covers

Now that we have looked below your program to uncover the underlying software, let's open the covers of the computer to learn about the underlying hardware. The underlying hardware in any computer performs the same basic functions: inputting data, outputting data, processing data, and storing data. How these functions are performed is the primary topic of this book, and subsequent chapters deal with different parts of these four tasks. When we come to an important point in this book, a point so important that we hope you will remember it forever, we emphasize it by identifying it as a "Big Picture" item. We have about a dozen Big Pictures in this book, with the first being the five components of a computer that perform the tasks of inputting, outputting, processing, and storing data.

The five classic components of a computer are input, output, memory, datapath, and control, with the last two sometimes combined and called the processor. Figure 1.5 shows the standard organization of a computer. This organization is independent of hardware technology: You can place every piece of every computer, past and present, into one of these five categories. To help you keep all this in perspective, the five components of a computer are shown on the front page of the following chapters, with the portion of interest to that chapter highlighted.

## The BIG Picture

Figure 1.6 shows a typical desktop computer with keyboard, mouse, screen, and a box containing even more hardware. What is not visible in the photograph is a network that connects the computer to other computers. This photograph reveals two of the key components of computers: **input devices**, such as the keyboard and mouse, and **output devices**, such as the screen. As the names suggest, input feeds the computer, and output is the result of computation sent to the user. Some devices, such as networks and disks, provide both input and output to the computer.

**input device** A mechanism through which the computer receives information, such as the keyboard or mouse.

**output device** A mechanism that conveys the result of a computation to a user or another computer.





**FIGURE 1.8 Inside the personal computer of Figure 1.6 on page 17.** This packaging is sometimes called a clamshell because of the way it opens with hinges on one side. To see what's inside, let's start on the top left-hand side. The shiny metal box on the top far left side is the power supply. Just below that on the far left is the fan, with its cover pulled back. To the right and below the fan is a printed circuit board (PC board), called the *motherboard* in a PC, that contains most of the electronics of the computer; Figure 1.10 is a close-up of that board. The processor is the large raised rectangle just to the right of the fan. On the right side we see the bays designed to hold types of disk drives. The top bay contains a DVD drive, the middle bay a Zip drive, and the bottom bay contains a hard disk.

**motherboard** A plastic board containing packages of integrated circuits or chips, including processor, cache, memory, and connectors for I/O devices such as networks and disks.

**integrated circuit** Also called **chip**. A device combining dozens to millions of transistors.

**memory** The storage area in which programs are kept when they are running and that contains the data needed by the running programs.

**central processor unit (CPU)** Also called processor. The active part of the computer, which contains the datapath and control and which adds numbers, tests numbers, signals I/O devices to activate, and so on.

**datapath** The component of the processor that performs arithmetic operations.

**control** The component of the processor that commands the datapath, memory, and I/O devices according to the instructions of the program.

**dynamic random access memory (DRAM)** Memory built as an integrated circuit, it provides random access to any location.

**cache memory** A small, fast memory that acts as a buffer for a slower, larger memory.

shows the contents of the desktop computer in Figure 1.6. This **motherboard** is shown vertically on the left with the power supply. Three disk drives—a DVD drive, Zip drive, and hard drive—appear on the right.

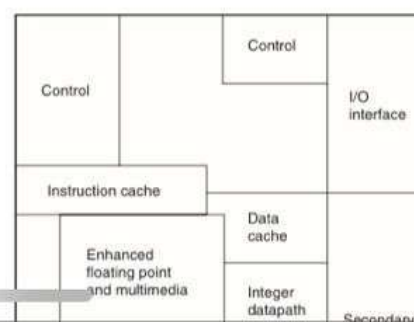
The small rectangles on the motherboard contain the devices that drive our advancing technology, **integrated circuits** or **chips**. The board is composed of three pieces: the piece connecting to the I/O devices mentioned earlier, the memory, and the processor. The I/O devices are connected via the two large boards attached perpendicularly to the motherboard toward the middle on the right-hand side.

The **memory** is where the programs are kept when they are running; it also contains the data needed by the running programs. In Figure 1.8, memory is found on the two small boards that are attached perpendicularly toward the middle of the motherboard. Each small memory board contains eight integrated circuits.

The **processor** is the active part of the board, following the instructions of a program to the letter. It adds numbers, tests numbers, signals I/O devices to activate, and so on. The processor is the large square below the memory boards in the lower-right corner of Figure 1.8. Occasionally, people call the processor the **CPU**, for the more bureaucratic-sounding **central processor unit**.

Descending even lower into the hardware, Figure 1.9 reveals details of the processor in Figure 1.8. The processor comprises two main components: datapath and control, the respective brawn and brain of the processor. The **datapath** performs the arithmetic operations, and **control** tells the datapath, memory, and I/O devices what to do according to the wishes of the instructions of the program. Chapter 5 explains the datapath and control for a straightforward implementation, and Chapter 6 describes the changes needed for a higher-performance design.

Descending into the depths of any component of the hardware reveals insights into the machine. The memory in Figure 1.10 is built from DRAM chips. **DRAM** stands for **dynamic random access memory**. Several DRAMs are used together to contain the instructions and data of a program. In contrast to sequential access memories such as magnetic tapes, the **RAM** portion of the term DRAM means that memory accesses take the same amount of time no matter what portion of the memory is read. Inside the processor is another type of memory—cache memory. **Cache memory** consists of a small, fast memory that acts as a buffer for the DRAM memory. (The nontechnical definition of *cache* is a safe place for hiding things.) Cache is built using a different memory technology, static random access memory (SRAM). SRAM is faster but less dense, and hence more expensive, than DRAM.



tens, if not a hundred, computers. Today, local area network technology offers a capacity of from 100 million bits per second to a gigabit per second, usually shared by at most a few computers. Furthermore, 10-gigabit technology is in development! Optical communications technology has allowed similar growth in the capacity of wide area networks from hundreds of kilobits to gigabits, and from hundreds of computers connected to a worldwide network to millions of computers connected. This combination of dramatic rise in deployment of networking

combined with the increases in capacity have made network technology central to the information revolution of the last 25 years.

Recently, another innovation in networking is reshaping the way computers communicate. Wireless technology has become widely deployed, and most laptops now incorporate this technology. The ability to make a radio in the same low-cost semiconductor technology (CMOS) used for memory and microprocessors enabled a significant improvement in price, leading to an explosion in deployment. Currently available wireless technologies, called by the IEEE standard name 802.11, allow for transmission rates from 1 to less than 100 million bits per second. Wireless technology is quite a bit different from wire-based networks, since all users in an immediate area share the airwaves.

1. Semiconductor DRAM and disk storage differ significantly. Describe the fundamental difference for each of the following: volatility, access time, and cost.

### Check Yourself

### Technologies for Building Processors and Memories

Processors and memory have improved at an incredible rate because computer designers have long embraced the latest in electronic technology to try to win the race to design a better computer. Figure 1.12 shows the technologies that have been used over time, with an estimate of the relative performance per unit cost for each technology. This section explores the technology that has fueled the computer industry since 1975 and will continue to do so for the foreseeable future. Since this technology shapes what computers will be able to do and how quickly they will evolve, we believe all computer professionals should be familiar with the basics of integrated circuits.

A **transistor** is simply an on/off switch controlled by electricity. The **integrated circuit (IC)** combined dozens to hundreds of transistors into a single chip. To describe the tremendous increase in the number of transistors from hundreds to

**transistor** An on/off switch controlled by an electric signal.

| Year | Technology used in computers         | Relative performance/unit cost |
|------|--------------------------------------|--------------------------------|
| 1951 | Vacuum tube                          | 1                              |
| 1965 | Transistor                           | 35                             |
| 1975 | Integrated circuit                   | 900                            |
| 1995 | Very large scale integrated circuit  | 2,400,000                      |
| 2005 | Ultra large scale integrated circuit | 6,200,000,000                  |

**FIGURE 1.12** Relative performance per unit cost of technologies used in computers over time. Source: Computer Museum, Boston, with 2005 extrapolated by the authors.

**vacuum tube** An electronic component, predecessor of the transistor, that consists of a hollow glass tube about 5 to 10 cm long from which as much air has been removed as possible and which uses an electron beam to transfer data.





Done

patterson\_book-CA.pdf



portability across computers.

hardware that obeys the architecture abstraction. These ideas bring us to another Big Picture.

im  
tl  
a

42 of 689

### A Safe Place for Data

Thus far we have seen how to input data, compute using the data, and display data. If we were to lose power to the computer, however, everything would be lost because the **memory** inside the computer is **volatile**—that is, when it loses power, it forgets. In contrast, a cassette tape for a stereo doesn't forget the recorded music when you turn off the power because the tape is magnetic and is thus a **nonvolatile memory** technology.

To distinguish between the memory used to hold programs while they are running and this nonvolatile memory used to store programs between runs, the term **primary memory** or **main memory** is used for the former, and **secondary memory** for the latter. DRAMs have dominated main memory since 1975, but **magnetic disks** have dominated secondary memory since 1965. In embedded applications, FLASH, a nonvolatile semiconductor memory is also used.

Today the primary nonvolatile storage used on all desktop and server computers is the magnetic hard disk. As Figure 1.11 shows, a magnetic hard disk consists of a collection of platters, which rotate on a spindle at 5400 to 15,000 revolutions per minute. The metal platters are covered with magnetic recording material on both sides, similar to the material found on a cassette or video tape. To read and write information on a hard disk, a movable **arm** containing a small electromagnetic coil called a **read/write head** is located just above each surface. The entire drive is permanently sealed to control the environment inside the drive, which, in turn, allows the disk heads to be much closer to the drive surface.

Diameters of hard disks vary by more than a factor of 3 today, from less than 1 inch to 3.5 inches, and have been shrunk over the years to fit into new products; workstation servers, personal computers, laptops, palmtops, and digital cameras have all inspired new disk form factors. Traditionally, the widest disks have the highest performance, the smallest disks have the lowest unit cost, and the best cost per **megabyte** is usually a disk in between. Although most hard drives appear inside computers (as in Figure 1.8), hard drives can also be attached using external interfaces such as Firewire or USB.

The use of mechanical components means that access times for magnetic disks are much slower than for DRAMs: disks typically take 5–15 milliseconds, while DRAMs take 40–80 nanoseconds—making DRAMs about 100,000 times faster. Yet disks have much lower costs than DRAM for the same storage capacity because the production costs for a given amount of disk storage are lower than for the same amount of integrated circuit. In 2004, the cost per megabyte of disk is about 100 times less expensive than DRAM.

Thus there are three primary differences between magnetic disks and main memory: disks are nonvolatile because they are magnetic; they have a slower access time because they are mechanical devices; and they are cheaper per megabyte because they have very high storage capacity at a modest cost.

**memory** The storage area in which programs are kept when they are running and that contains the data needed by the running programs.

**volatile memory** Storage, such as DRAM, that only retains data only if it is receiving power.

**nonvolatile memory** A form of memory that retains data even in the absence of a power source and that is used to store programs between runs. Magnetic disk is nonvolatile and DRAM is not.

**primary memory** Also called main memory. Volatile memory used to hold programs while they are running; typically consists of DRAM in today's computers.

**secondary memory** Nonvolatile memory used to store programs and data between runs; typically consists of magnetic disks in today's computers.

**magnetic disk (also called hard disk)** A form of nonvolatile secondary memory composed of rotating platters coated with a magnetic recording material.

**megabyte** Traditionally 1,048,576 ( $2^{20}$ ) bytes, although some communications and secondary storage systems have redefined it to mean 1,000,000 ( $10^6$ ) bytes.





## 2.1 Introduction

**instruction set** The vocabulary of commands understood by a given architecture.

To command a computer's hardware, you must speak its language. The words of a computer's language are called *instructions*, and its vocabulary is called an **instruction set**. In this chapter, you will see the instruction set of a real computer, both in the form written by humans and in the form read by the computer. We introduce instructions in a top-down fashion. Starting from a notation that looks like a restricted programming language, we refine it step-by-step until you see the real language of a real computer. Chapter 3 continues our downward descent, unveiling the representation of integer and floating-point numbers and the hardware that operates on them.

You might think that the languages of computers would be as diverse as those of humans, but in reality computer languages are quite similar, more like regional dialects than like independent languages. Hence, once you learn one, it is easy to pick up others. This similarity occurs because all computers are constructed from hardware technologies based on similar underlying principles and because there are a few basic operations that all computers must provide. Moreover, computer designers have a common goal: to find a language that makes it easy to build the hardware and the compiler while maximizing performance and minimizing cost. This goal is time-honored; the following quote was written before you could buy a computer, and it is as true today as it was in 1947:

*It is easy to see by formal-logical methods that there exist certain [instruction sets] that are in abstract adequate to control and cause the execution of any sequence of operations. . . . The really decisive considerations from the present point of view, in selecting an [instruction set], are more of a practical nature: simplicity of the equipment demanded by the [instruction set], and the clarity of its application to the actually important problems together with the speed of its handling of those problems.*

Burks, Goldstine, and von Neumann, 1947

The "simplicity of the equipment" is as valuable a consideration for computers of the 2000s as it was for those of the 1950s. The goal of this chapter is to teach an instruction set that follows this advice, showing both how it is represented in hardware and the relationship between high-level programming languages and this more primitive one. Our examples are in the C programming language; Section 2.14 shows how these would change for an object-oriented language like Java.

## 2.2 Operations of the Computer Hardware

49

By learning how to represent instructions, you will also discover the secret of computing: the **stored-program concept**. Moreover you will exercise your "foreign language" skills by writing programs in the language of the computer and running them on the simulator that comes with this book. You will also see the impact of programming languages and compiler optimization on performance. We conclude with a look at the historical evolution of instruction sets and an overview of other computer dialects.

The chosen instruction set comes from MIPS, which is typical of instruction sets designed since the 1980s. Almost 100 million of these popular microprocessors were manufactured in 2002, and they are found in products from ATI Technologies, Broadcom, Cisco, NEC, Nintendo, Silicon Graphics, Sony, Texas Instruments, and Toshiba, among others.

We reveal the MIPS instruction set a piece at a time, giving the rationale along with the computer structures. This top-down, step-by-step tutorial weaves the components with their explanations, making assembly language more palatable. To keep the overall picture in mind, each section ends with a figure summarizing the MIPS instruction set revealed thus far, highlighting the portions presented in that section.

**stored-program concept** The idea that instructions and data of many types can be stored in memory as numbers, leading to the stored program computer.

69 of 689

## 2.2 Operations of the Computer Hardware

Every computer must be able to perform arithmetic. The MIPS assembly language notation

```
add a, b, c
```

instructs a computer to add the two variables *b* and *c* and to put their sum in *a*.

This notation is rigid in that each MIPS arithmetic instruction performs only one operation and must always have exactly three variables. For example, suppose we want to place the sum of variables *b*, *c*, *d*, and *e* into variable *a*. (In this section we are being deliberately vague about what a "variable" is; in the next section we'll explain in detail.)

The following sequence of instructions adds the four variables:

```
add a, b, c      # The sum of b and c is placed in a.
add a, a, d      # The sum of b, c, and d is now in a.
add a, a, e      # The sum of b, c, d, and e is now in a.
```

*There must certainly be instructions for performing the fundamental arithmetic operations.*

Burks, Goldstine, and von Neumann, 1947

Thus, it takes three instructions to take the sum of four variables.

The words to the right of the sharp symbol (#) on each line above are *comments* for the human reader, and the computer ignores them. Note that unlike other programming languages, each line of this language can contain at most one instruction. Another difference from C is that comments always terminate at the end of a line.

The natural number of operands for an operation like addition is three: the two numbers being added together and a place to put the sum. Requiring every instruction to have exactly three operands, no more and no less, conforms to the philosophy of keeping the hardware simple: hardware for a variable number of operands is more complicated than hardware for a fixed number. This situation illustrates the first of four underlying principles of hardware design:

*Design Principle 1: Simplicity favors regularity.*

We can now show, in the two examples that follow, the relationship of programs written in higher-level programming languages to programs in this more primitive notation.

## EXAMPLE

## Compiling Two C Assignment Statements into MIPS

This segment of a C program contains the five variables *a*, *b*, *c*, *d*, and *e*. Since Java evolved from C, this example and the next few work for either high-level programming language:

```
a = b + c;
d = a - e;
```

The translation from C to MIPS assembly language instructions is performed by the *compiler*. Show the MIPS code produced by a compiler.

## ANSWER

A MIPS instruction operates on two source operands and places the result in one destination operand. Hence, the two simple statements above compile directly into these two MIPS assembly language instructions:

```
add a, b, c
sub d, a, e
```



**Compiling a Complex C Assignment into MIPS**

A somewhat complex statement contains the five variables  $f$ ,  $g$ ,  $h$ ,  $i$ , and  $j$ :

$$f = (g + h) - (i + j);$$

What might a C compiler produce?

**EXAMPLE**

The compiler must break this statement into several assembly instructions since only one operation is performed per MIPS instruction. The first MIPS instruction calculates the sum of  $g$  and  $h$ . We must place the result somewhere, so the compiler creates a temporary variable, called  $t0$ :

```
add t0,g,h # temporary variable t0 contains g + h
```

Although the next operation is subtract, we need to calculate the sum of  $i$  and  $j$  before we can subtract. Thus, the second instruction places the sum  $i$  and  $j$  in another temporary variable created by the compiler, called  $t1$ :

```
add t1,i,j # temporary variable t1 contains i + j
```

Finally, the subtract instruction subtracts the second sum from the first and places the difference in the variable  $f$ , completing the compiled code:

```
sub f,t0,t1 # f gets t0 - t1, which is (g + h) - (i + j)
```

**ANSWER**

Figure 2.1 summarizes the portions of MIPS assembly language described in this section. These instructions are symbolic representations of what the MIPS processor actually understands. In the next few sections, we will evolve this symbolic representation into the real language of MIPS, with each step making the symbolic representation more concrete.

**MIPS assembly language**

| Category   | Instruction | Example   | Meaning     | Comments              |
|------------|-------------|-----------|-------------|-----------------------|
| Arithmetic | add         | add a,b,c | $a = b + c$ | Always three operands |
|            | subtract    | sub a,b,c | $a = b - c$ | Always three operands |

**FIGURE 2.1 MIPS architecture revealed in Section 2.2.** The real computer operands will be unveiled in the next section. Highlighted portions in such summaries show MIPS assembly language structures introduced in this section; for this first figure, all is new.

**Check Yourself**

For a given function, which programming language likely takes the most lines of code? Put the three representations below in order.

1. Java
2. C
3. MIPS assembly language

**Elaboration:** To increase portability, Java was originally envisioned as relying on a software interpreter. The instruction set of this interpreter is called *Java bytecodes*, which is quite different from the MIPS instruction set. To get performance close to the equivalent C program, Java systems today typically compile Java bytecodes into the native instruction sets like MIPS. Because this compilation is normally done much later than for C programs, such Java compilers are often called *Just-In-Time (JIT)* compilers. Section 2.10 shows how JITs are used later than C compilers in the startup process, and Section 2.13 shows the performance consequences of compiling versus interpreting Java programs. The Java examples in this chapter skip the Java bytecode step and just show the MIPS code that are produced by a compiler.

Section 2.12 shows how `args` are used later than a compiler in the startup process, and Section 2.13 shows the performance consequences of compiling versus interpreting Java programs. The Java examples in this chapter skip the Java bytecode step and just show the MIPS code that are produced by a compiler.

72 of 689

## 2.3

## Operands of the Computer Hardware

Unlike programs in high-level languages, the operands of arithmetic instructions are restricted; they must be from a limited number of special locations built directly in hardware called *registers*. Registers are the bricks of computer construction: registers are primitives used in hardware design that are also visible to the programmer when the computer is completed. The size of a register in the MIPS architecture is 32 bits; groups of 32 bits occur so frequently that they are given the name **word** in the MIPS architecture.

**word** The natural unit of access in a computer, usually a group of 32 bits; corresponds to the size of a register in the MIPS architecture.

One major difference between the variables of a programming language and registers is the limited number of registers, typically 32 on current computers. MIPS has 32 registers. (See Section 2.19 for the history of the number of registers.) Thus, continuing in our top-down, stepwise evolution of the symbolic representation of the MIPS language, in this section we have added the restriction that the three operands of MIPS arithmetic instructions must each be chosen from one of the 32 32-bit registers.

The reason for the limit of 32 registers may be found in the second of our four underlying design principles of hardware technology:

## 2.3 Operands of the Computer Hardware

53

*Design Principle 2: Smaller is faster.*

A very large number of registers may increase the clock cycle time simply because it takes electronic signals longer when they must travel farther.

Guidelines such as “smaller is faster” are not absolutes; 31 registers may not be faster than 32. Yet, the truth behind such observations causes computer designers to take them seriously. In this case, the designer must balance the craving of programs for more registers with the designer’s desire to keep the clock cycle fast. Another reason for not using more than 32 is the number of bits it would take in the instruction format, as Section 2.4 demonstrates.

Chapters 5 and 6 show the central role that registers play in hardware construction; as we shall see in this chapter, effective use of registers is key to program performance.

Although we could simply write instructions using numbers for registers, from 0 to 31, the MIPS convention is to use two-character names following a dollar sign to represent a register. Section 2.7 will explain the reasons behind these names. For now, we will use `$s0, $s1, ...` for registers that correspond to variables in C and Java programs and `$t0, $t1, ...` for temporary registers needed to compile the program into MIPS instructions.

## Compiling a C Assignment Using Registers

It is the compiler’s job to associate program variables with registers. Take, for instance, the assignment statement from our earlier example:

```
f = (g + h) - (i + j);
```

The variables `f`, `g`, `h`, `i`, and `j` are assigned to the registers `$s0`, `$s1`, `$s2`, `$s3`, and `$s4`, respectively. What is the compiled MIPS code?

The compiled program is very similar to the prior example, except we replace the variables with the register names mentioned above plus two temporary registers, `$t0` and `$t1`, which correspond to the temporary variables above:

```
add $t0,$s1,$s2 # register $t0 contains g + h
add $t1,$s3,$s4 # register $t1 contains i + j
sub $s0,$t0,$t1 # f gets $t0 - $t1, which is (g + h) - (i + j)
```

## EXAMPLE

## ANSWER

We are now ready to explain the difference between the way humans instruct computers and the way computers see instructions. First, let's quickly review how a computer represents numbers.

Humans are taught to think in base 10, but numbers may be represented in any base. For example, 123 base 10 = 1111011 base 2.

Numbers are kept in computer hardware as a series of high and low electronic signals, and so they are considered base 2 numbers. (Just as base 10 numbers are called *decimal* numbers, base 2 numbers are called *binary* numbers.) A single digit of a binary number is thus the "atom" of computing, since all information is composed of **binary digits** or *bits*. This fundamental building block can be one of two values, which can be thought of as several alternatives: high or low, on or off, true or false, or 1 or 0.

Instructions are also kept in the computer as a series of high and low electronic signals and may be represented as numbers. In fact, each piece of an instruction can be considered as an individual number, and placing these numbers side by side forms the instruction.

Since registers are part of almost all instructions, there must be a convention to map register names into numbers. In MIPS assembly language, registers \$s0 to \$s7 map onto registers 16 to 23, and registers \$t0 to \$t7 map onto registers 8 to 15. Hence, \$s0 means register 16, \$s1 means register 17, \$s2 means register 18, ..., \$t0 means register 8, \$t1 means register 9, and so on. We'll describe the convention for the rest of the 32 registers in the following sections.

**binary digit** Also called binary bit. One of the two numbers in base 2, 0 or 1, that are the components of information.

### Translating a MIPS Assembly Instruction into a Machine Instruction

Let's do the next step in the refinement of the MIPS language as an example. We'll show the real MIPS language version of the instruction represented symbolically as

```
add $t0,$s1,$s2
```

first as a combination of decimal numbers and then of binary numbers.

The decimal representation is

|   |    |    |   |   |    |
|---|----|----|---|---|----|
| 0 | 17 | 18 | 8 | 0 | 32 |
|---|----|----|---|---|----|

Each of these segments of an instruction is called a *field*. The first and last fields (containing 0 and 32 in this case) in combination tell the MIPS computer that this instruction performs addition. The second field gives the number of the register that is the first source operand of the addition operation (17 = \$s1), and the third field gives the other source operand for the addition (18 = \$s2). The fourth field contains the number of the register that is to receive the sum (8 = \$t0). The fifth field is unused in this instruction, so it is set to 0. Thus, this instruction adds register \$s1 to register \$s2 and places the sum in register \$t0.

This instruction can also be represented as fields of binary numbers as opposed to decimal:

|        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|
| 000000 | 10001  | 10010  | 01000  | 00000  | 100000 |
| 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits |

To distinguish it from assembly language, we call the numeric version of instructions **machine language** and a sequence of such instructions *machine code*.

This layout of the instruction is called the **instruction format**. As you can see from counting the number of bits, this MIPS instruction takes exactly 32 bits—the same size as a data word. In keeping with our design principle that simplicity favors regularity, all MIPS instructions are 32 bits long.

It would appear that you would now be reading and writing long, tedious strings of binary numbers. We avoid that tedium by using a higher base than binary that con-

### EXAMPLE

### ANSWER

**machine language** Binary representation used for communication within a computer system.

**instruction format** A form of representation of an instruction composed of fields of binary numbers.



## patterson\_book-CA.pdf

82 of 689

## EXAMPLE

## ANSWER

Because we frequently deal with different number bases, to avoid confusion we will subscript decimal numbers with *ten*, binary numbers with *two*, and hexadecimal numbers with *hex*. (If there is no subscript, the default is base 10.) By the way, C and Java use the notation *0xnnnn* for hexadecimal numbers.

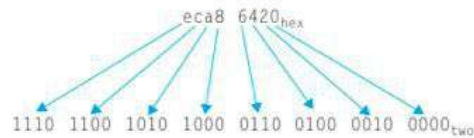
## Binary-to-Hexadecimal and Back

Convert the following hexadecimal and binary numbers into the other base:

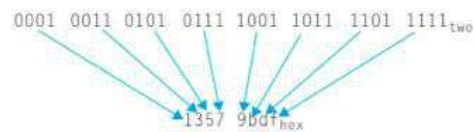
$\text{eca8 } 6420_{\text{hex}}$

$0001 \ 0011 \ 0101 \ 0111 \ 1001 \ 1011 \ 1101 \ 1111_{\text{two}}$

Just a table lookup one way:



And then the other direction too:



## MIPS Fields

MIPS fields are given names to make them easier to discuss:

| op     | rs     | rt     | rd     | shamt  | funct  |
|--------|--------|--------|--------|--------|--------|
| 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits |

Here is the meaning of each name of the fields in MIPS instructions:

- *op*: Basic operation of the instruction, traditionally called the **opcode**.
- *rs*: The first register source operand.
- *rt*: The second register source operand.
- *rd*: The register destination operand. It gets the result of the operation.
- *shamt*: Shift amount. (Section 2.5 explains shift instructions and this term; it will not be used until then, and hence the field contains zero.)
- *funct*: Function. This field selects the specific variant of the operation in the *op* field and is sometimes called the **function code**.

**opcode** The field that denotes the operation and format of an instruction.

A problem occurs when an instruction needs longer fields than those shown above. For example, the load word instruction must specify two registers and a constant. If the address were to use one of the 5-bit fields in the format above, the constant within the load word instruction would be limited to only  $2^5$  or 32. This constant is used to select elements from arrays or data structures, and it often needs to be much larger than 32. This 5-bit field is too small to be useful.

Hence, we have a conflict between the desire to keep all instructions the same length and the desire to have a single instruction format. This leads us to the final hardware design principle:

*Design Principle 4:* Good design demands good compromises.

The compromise chosen by the MIPS designers is to keep all instructions the same length, thereby requiring different kinds of instruction formats for different kinds of instructions. For example, the format above is called *R-type* (for register) or *R-format*. A second type of instruction format is called *I-type* (for immediate) or *I-format* and is used by the immediate and data transfer instructions. The fields of I-format are

| op     | rs     | rt     | constant or address |
|--------|--------|--------|---------------------|
| 6 bits | 5 bits | 5 bits | 16 bits             |

**Translating MIPS Assembly Language into Machine Language**

We can now take an example all the way from what the programmer writes to what the computer executes. If `$t1` has the base of the array `A` and `$s2` corresponds to `h`, the assignment statement

```
A[300] = h + A[300];
```

is compiled into

```
lw    $t0,1200($t1) # Temporary reg $t0 gets A[300]
add   $t0,$s2,$t0    # Temporary reg $t0 gets h + A[300]
sw    $t0,1200($t1) # Stores h + A[300] back into A[300]
```

What is the MIPS machine language code for these three instructions?

For convenience, let's first represent the machine language instructions using decimal numbers. From Figure 2.6, we can determine the three machine language instructions:

| op | ra | rt | rd | address/<br>shamt | funct |
|----|----|----|----|-------------------|-------|
| 35 | 9  | 8  |    | 1200              |       |
| 0  | 18 | 8  | 8  | 0                 | 32    |
| 43 | 9  | 8  |    | 1200              |       |

The `lw` instruction is identified by 35 (see Figure 2.6) in the first field (op). The base register 9 (`$t1`) is specified in the second field (rs), and the destination register 8 (`$t0`) is specified in the third field (rt). The offset to select `A[300]` ( $1200 = 300 \times 4$ ) is found in the final field (address).

The `add` instruction that follows is specified with 0 in the first field (op) and 32 in the last field (funct). The three register operands (18, 8, and 8) are found in the second, third, and fourth fields and correspond to `$s2`, `$t0`, and `$t0`.

**EXAMPLE****ANSWER**

The `sw` instruction is identified with 43 in the first field. The rest of this final instruction is identical to the `lw` instruction.

The binary equivalent to the decimal form is the following (1200 in base 10 is 0000 0100 1011 0000 base 2):

|        |       |       |                     |
|--------|-------|-------|---------------------|
| 10011  | 01001 | 01000 | 0000 0100 1011 0000 |
| 000000 | 10010 | 01000 | 01000 00000 100000  |
| 101011 | 01001 | 01000 | 0000 0100 1011 0000 |

Note the similarity of the binary representations of the first and last instructions. The only difference is in the third bit from the left.

Figure 2.7 summarizes the portions of MIPS assembly language described in this section. As we shall see in Chapters 5 and 6, the similarity of the binary representations of related instructions simplifies hardware design. These instructions are another example of regularity in the MIPS architecture.

**Check Yourself**

Why doesn't MIPS have a subtract immediate instruction?

1. Negative constants appear much less frequently in C and Java, so they are not the common case and do not merit special support.
2. Since the immediate field holds both negative and positive constants, add immediate with a negative number is equivalent to subtract immediate with a positive number, so subtract immediate is superfluous.

## 2.4 Representing Instructions in the Computer

## MIPS operands

| Name                  | Example                                                | Comments                                                                                                                                                                              |
|-----------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 32 registers          | $\$s0, \$s1, \dots, \$s7$<br>$\$t0, \$t1, \dots, \$t7$ | Fast locations for data. In MIPS, data must be in registers to perform arithmetic. Registers $\$s0$ - $\$s7$ map to 16-23 and $\$t0$ - $\$t7$ map to 8-15.                            |
| $2^{30}$ memory words | Memory[0],<br>Memory[4], ...,<br>Memory[4294967292]    | Accessed only by data transfer instructions in MIPS. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers. |

## MIPS assembly language

| Category      | Instruction | Example                | Meaning                            | Comments                          |
|---------------|-------------|------------------------|------------------------------------|-----------------------------------|
| Arithmetic    | add         | add $\$s1, \$s2, \$s3$ | $\$s1 = \$s2 + \$s3$               | Three operands; data in registers |
|               | subtract    | sub $\$s1, \$s2, \$s3$ | $\$s1 = \$s2 - \$s3$               | Three operands; data in registers |
| Data transfer | load word   | lw $\$s1, 100(\$s2)$   | $\$s1 = \text{Memory}[\$s2 + 100]$ | Data from memory to register      |
|               | store word  | sw $\$s1, 100(\$s2)$   | $\text{Memory}[\$s2 + 100] = \$s1$ | Data from register to memory      |

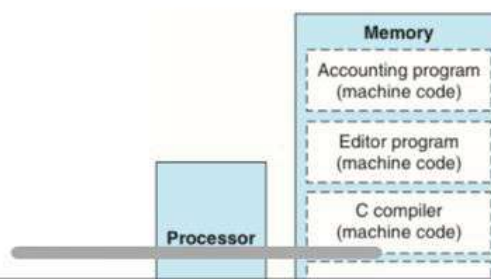
## MIPS machine language

| Name       | Format | Example |        |        |        |         |        | Comments                      |
|------------|--------|---------|--------|--------|--------|---------|--------|-------------------------------|
| add        | R      | 0       | 18     | 19     | 17     | 0       | 32     | add $\$s1, \$s2, \$s3$        |
| sub        | R      | 0       | 18     | 19     | 17     | 0       | 34     | sub $\$s1, \$s2, \$s3$        |
| addi       | I      | 8       | 18     | 17     |        | 100     |        | addi $\$s1, \$s2, 100$        |
| lw         | I      | 35      | 18     | 17     |        | 100     |        | lw $\$s1, 100(\$s2)$          |
| sw         | I      | 43      | 18     | 17     |        | 100     |        | sw $\$s1, 100(\$s2)$          |
| Field size |        | 6 bits  | 5 bits | 5 bits | 5 bits | 5 bits  | 6 bits | All MIPS instructions 32 bits |
| R-format   | R      | op      | rs     | rt     | rd     | shamt   | funct  | Arithmetic instruction format |
| I-format   | I      | op      | rs     | rt     |        | address |        | Data transfer format          |

**FIGURE 2.7 MIPS architecture revealed through Section 2.4.** Highlighted portions show MIPS machine language structures introduced in Section 2.4. The two MIPS instruction formats so far are R and I. The first 16 bits are the same: both contain an *op* field, giving the base operation; an *rs* field, giving one of the sources; and the *rt* field, which specifies the other source operand, except for load word, where it specifies the destination register. R-format divides the last 16 bits into an *rd* field, specifying the destination register; *shamt* field, which Section 2.5 explains; and the *funct* field, which specifies the specific operation of R-format instructions. I-format keeps the last 16 bits as a single *address* field.

**Elaboration:** Representing decimal numbers in base 2 gives an easy way to represent positive integers in computer words. Chapter 3 explains how to represent negative numbers, but for now take it on faith that a 32-bit word can represent integers between  $-2^{31}$  and  $+2^{31} - 1$  or -2,147,483,648 to +2,147,483,647, and the 16-bit constant field really holds  $-2^{15}$  to  $+2^{15} - 1$  or -32,768 to 32,767. Such integers are called *two's complement* numbers. Chapter 3 shows how we would encode `addi $t0, $t0, -1` or `lw $t0, -4($s0)`, which require negative numbers in the constant field of the immediate format.

## Chapter 2 Instructions: Language of the Computer





Done

patterson\_book-CA.pdf



90 of 689

2.6 Logical operations

is this value in register \$t0:

```
1111 1111 1111 1111 1100 0011 1111 1111two
```

Figure 2.9 above shows the relationship between the C and Java operators and the MIPS instructions. Constants are useful in AND and OR logical operations as well as in arithmetic operations, so MIPS also provides the instructions *and immediate* (`andi`) and *or immediate* (`ori`). Constants are rare for NOR, since its main use is to invert the bits of a single operand; thus, the hardware has no immediate version. Figure 2.10, which summarizes the MIPS instructions seen thus far, highlights the logical instructions.

## MIPS operands

| Name                  | Example                                                                                     | Comments                                                                                                                                                                      |
|-----------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 32 registers          | <code>\$s0, \$s1, ..., \$s7</code><br><code>\$t0, \$t1, ..., \$t7</code>                    | Fast locations for data. In MIPS, data must be in registers to perform arithmetic. Registers <code>\$s0-\$s7</code> map to 16–23 and <code>\$t0-\$t7</code> map to 8–15.      |
| $2^{30}$ memory words | <code>Memory[0]</code> ,<br><code>Memory[4]</code> , ...<br><code>Memory[4294967292]</code> | Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers. |

## MIPS assembly language

| Category      | Instruction                      | Example                           | Meaning                                | Comments                            |
|---------------|----------------------------------|-----------------------------------|----------------------------------------|-------------------------------------|
| Arithmetic    | <code>add</code>                 | <code>add \$s1, \$s2, \$s3</code> | <code>\$s1 = \$s2 + \$s3</code>        | Three operands; overflow detected   |
|               | <code>subtract</code>            | <code>sub \$s1, \$s2, \$s3</code> | <code>\$s1 = \$s2 - \$s3</code>        | Three operands; overflow detected   |
|               | <code>add immediate</code>       | <code>addi \$s1, \$s2, 100</code> | <code>\$s1 = \$s2 + 100</code>         | + constant; overflow detected       |
| Logical       | <code>and</code>                 | <code>and \$s1, \$s2, \$s3</code> | <code>\$s1 = \$s2 &amp; \$s3</code>    | Three reg. operands; bit-by-bit AND |
|               | <code>or</code>                  | <code>or \$s1, \$s2, \$s3</code>  | <code>\$s1 = \$s2   \$s3</code>        | Three reg. operands; bit-by-bit OR  |
|               | <code>nor</code>                 | <code>nor \$s1, \$s2, \$s3</code> | <code>\$s1 = ~( \$s2   \$s3 )</code>   | Three reg. operands; bit-by-bit NOR |
|               | <code>and immediate</code>       | <code>andi \$s1, \$s2, 100</code> | <code>\$s1 = \$s2 &amp; 100</code>     | Bit-by-bit AND reg with constant    |
|               | <code>or immediate</code>        | <code>ori \$s1, \$s2, 100</code>  | <code>\$s1 = \$s2   100</code>         | Bit-by-bit OR reg with constant     |
|               | <code>shift left logical</code>  | <code>sll \$s1, \$s2, 10</code>   | <code>\$s1 = \$s2 &lt;&lt; 10</code>   | Shift left by constant              |
| Data transfer | <code>shift right logical</code> | <code>srl \$s1, \$s2, 10</code>   | <code>\$s1 = \$s2 &gt;&gt; 10</code>   | Shift right by constant             |
|               | <code>load word</code>           | <code>lw \$s1, 100(\$s2)</code>   | <code>\$s1 = Memory[\$s2 + 100]</code> | Word from memory to register        |
|               | <code>store word</code>          | <code>sw \$s1, 100(\$s2)</code>   | <code>Memory[\$s2 + 100] = \$s1</code> | Word from register to memory        |

**FIGURE 2.10 MIPS architecture revealed thus far.** Color indicates the portions introduced since Figure 2.7 on page 67. The back endpapers of this book also list the MIPS machine language.

The utility of an automatic computer lies in the possibility of using a given sequence of instructions repeatedly, the number of times it is repeated

## 2.6

## Instructions for Making Decisions



Done

patterson\_book-CA.pdf



The utility of an automatic computer lies in the possibility of

91 of 689

When the iteration is completed a different sequence of [instructions] is to be followed, so we must, in most cases, give two parallel trains of [instructions] preceded by an instruction as to which routine is to be followed. This choice can be made to depend upon the sign of a number (zero being reckoned as plus for machine purposes). Consequently, we introduce an [instruction] (the conditional transfer [instruction]) which will, depending on the sign of a given number, cause the proper one of two routines to be executed.

Burks, Goldstine, and von Neumann, 1947

## EXAMPLE

## ANSWER

## 2.6 Instructions for Making Decisions

What distinguishes a computer from a simple calculator is its ability to make decisions. Based on the input data and the values created during computation, different instructions execute. Decision making is commonly represented in programming languages using the *if* statement, sometimes combined with *go to* statements and labels. MIPS assembly language includes two decision-making instructions, similar to an *if* statement with a *go to*. The first instruction is

```
beq register1, register2, L1
```

This instruction means go to the statement labeled L1 if the value in register1 equals the value in register2. The mnemonic *beq* stands for *branch if equal*. The second instruction is

```
bne register1, register2, L1
```

It means go to the statement labeled L1 if the value in register1 does not equal the value in register2. The mnemonic *bne* stands for *branch if not equal*. These two instructions are traditionally called **conditional branches**.

## Compiling if-then-else into Conditional Branches

In the following code segment, *f*, *g*, *h*, *i*, and *j* are variables. If the five variables *f* through *j* correspond to the five registers \$s0 through \$s4, what is the compiled MIPS code for this C *if* statement?

```
if (i == j) f = g + h; else f = g - h;
```

Figure 2.11 is a flowchart of what the MIPS code should do. The first expression compares for equality, so it would seem that we would want *beq*. In general, the code will be more efficient if we test for the opposite condition to branch over the code that performs the subsequent *then* part of the *if* (the label *Else* is defined below).

```
bne $s3,$s4,Else # go to Else if i != j
```

## 2.6 Instructions for Making Decisions

73

The next assignment statement performs a single operation, and if all the operands are allocated to registers, it is just one instruction:

```
add $s0,$s1,$s2 # f = g + h (skipped if i != j)
```

We now need to go to the end of the *if* statement. This example introduces another kind of branch, often called an *unconditional branch*. This instruction says that the processor always follows the branch. To distinguish between conditional and unconditional branches, the MIPS name for this type of instruction is *jump*, abbreviated as *j* (the label *Exit* is defined below).

```
j Exit # go to Exit
```

The assignment statement in the *else* portion of the *if* statement can again be compiled into a single instruction. We just need to append the label *Else* to this instruction. We also show the label *Exit* that is after this instruction, showing the end of the *if-then-else* compiled code:

```
Else: sub $s0,$s1,$s2 # f = g - h (skipped if i == j)
Exit:
```

**conditional branch** An instruction that requires the comparison of two values and that allows for a subsequent transfer of control to a new address in the program based on the outcome of the comparison.

Notice that the assembler relieves the compiler and the assembly language programmer from the tedium of calculating addresses for branches, just as it does for calculating data addresses for loads and stores (see Section 2.10).

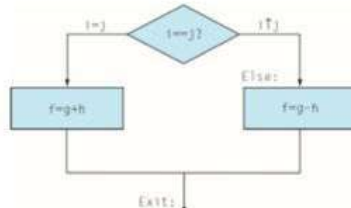


FIGURE 2.11 Illustration of the options in the *if* statement above. The left box corresponds to the *then* part of the *if* statement, and the right box corresponds to the *else* part.

## patterson\_book-CA.pdf

**ELABORATION.** If you have heard about delayed branches, covered in Chapter 6, don't worry: The MIPS assembler makes them invisible to the assembly language programmer.

96 of 689

## 2.6 Instructions for Making Decisions

77

## MIPS operands

| Name                         | Example                                                                                          | Comments                                                                                                                                                                                                                    |
|------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 32 registers                 | <code>\$s0, \$s1, ..., \$s7</code><br><code>\$t0, \$t1, ..., \$t7,</code><br><code>\$zero</code> | Fast locations for data. In MIPS, data must be in registers to perform arithmetic. Registers <code>\$s0–\$s7</code> map to 16–23 and <code>\$t0–\$t7</code> map to 8–15. MIPS register <code>\$zero</code> always equals 0. |
| 2 <sup>30</sup> memory words | <code>Memory(0),</code><br><code>Memory(4), ...,</code><br><code>Memory(4294967292)</code>       | Accessed only by data transfer instructions in MIPS. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers.                                       |

## MIPS assembly language

| Category           | Instruction                | Example                           | Meaning                                                                                 | Comments                                        |
|--------------------|----------------------------|-----------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------|
| Arithmetic         | add                        | add <code>\$s1, \$s2, \$s3</code> | <code>\$s1 = \$s2 + \$s3</code>                                                         | Three operands; data in registers               |
|                    | subtract                   | sub <code>\$s1, \$s2, \$s3</code> | <code>\$s1 = \$s2 - \$s3</code>                                                         | Three operands; data in registers               |
| Data transfer      | load word                  | lw <code>\$s1, 100(\$s2)</code>   | <code>\$s1 = Memory[\$s2 + 100]</code>                                                  | Data from memory to register                    |
|                    | store word                 | sw <code>\$s1, 100(\$s2)</code>   | <code>Memory[\$s2 + 100] = \$s1</code>                                                  | Data from register to memory                    |
| Logical            | and                        | and <code>\$s1, \$s2, \$s3</code> | <code>\$s1 = \$s2 &amp; \$s3</code>                                                     | Three reg. operands; bit-by-bit AND             |
|                    | or                         | or <code>\$s1, \$s2, \$s3</code>  | <code>\$s1 = \$s2   \$s3</code>                                                         | Three reg. operands; bit-by-bit OR              |
|                    | nor                        | nor <code>\$s1, \$s2, \$s3</code> | <code>\$s1 = ~( \$s2   \$s3 )</code>                                                    | Three reg. operands; bit-by-bit NOR             |
|                    | and immediate              | andi <code>\$s1, \$s2, 100</code> | <code>\$s1 = \$s2 &amp; 100</code>                                                      | Bit-by-bit AND reg with constant                |
|                    | or immediate               | ori <code>\$s1, \$s2, 100</code>  | <code>\$s1 = \$s2   100</code>                                                          | Bit-by-bit OR reg with constant                 |
|                    | shift left logical         | sll <code>\$s1, \$s2, 10</code>   | <code>\$s1 = \$s2 &lt;&lt; 10</code>                                                    | Shift left by constant                          |
|                    | shift right logical        | srl <code>\$s1, \$s2, 10</code>   | <code>\$s1 = \$s2 &gt;&gt; 10</code>                                                    | Shift right by constant                         |
| Conditional branch | branch on equal            | beq <code>\$s1, \$s2, L</code>    | if ( <code>\$s1 == \$s2</code> ) go to L                                                | Equal test and branch                           |
|                    | branch on not equal        | bne <code>\$s1, \$s2, L</code>    | if ( <code>\$s1 != \$s2</code> ) go to L                                                | Not equal test and branch                       |
|                    | set on less than           | slt <code>\$s1, \$s2, \$s3</code> | if ( <code>\$s2 &lt; \$s3</code> ) <code>\$s1 = 1;</code><br>else <code>\$s1 = 0</code> | Compare less than; used with beq, bne           |
|                    | set on less than immediate | sli <code>\$s1, \$s2, 100</code>  | if ( <code>\$s2 &lt; 100</code> ) <code>\$s1 = 1;</code><br>else <code>\$s1 = 0</code>  | Compare less than immediate; used with beq, bne |
| Unconditional jump | jump                       | j L                               | go to L                                                                                 | Jump to target address                          |

FIGURE 2.12 MIPS architecture revealed through Section 2.6. Highlighted portions show MIPS structures introduced in Section 2.6.

C has many statements for decisions and loops while MIPS has few. Which of the following do or do not explain this imbalance? Why?

**Check Yourself**

1. More decision statements make code easier to read and understand.
2. Fewer decision statements simplify the task of the underlying layer that is responsible for execution.

## MIPS machine language

| Name | Format | Example |    |    |     |    |    | Comments                          |
|------|--------|---------|----|----|-----|----|----|-----------------------------------|
| add  | R      | 0       | 18 | 19 | 17  | 0  | 32 | add <code>\$s1, \$s2, \$s3</code> |
| sub  | R      | 0       | 18 | 19 | 17  | 0  | 34 | sub <code>\$s1, \$s2, \$s3</code> |
| lw   | I      | 35      | 18 | 17 | 100 |    |    | lw <code>\$s1, 100(\$s2)</code>   |
| sw   | I      | 43      | 18 | 17 | 100 |    |    | sw <code>\$s1, 100(\$s2)</code>   |
| and  | R      | 0       | 18 | 19 | 17  | 0  | 36 | and <code>\$s1, \$s2, \$s3</code> |
| or   | R      | 0       | 18 | 19 | 17  | 0  | 37 | or <code>\$s1, \$s2, \$s3</code>  |
| nor  | R      | 0       | 18 | 19 | 17  | 0  | 39 | nor <code>\$s1, \$s2, \$s3</code> |
| andi | I      | 12      | 18 | 17 | 100 |    |    | andi <code>\$s1, \$s2, 100</code> |
| ori  | I      | 13      | 18 | 17 | 100 |    |    | ori <code>\$s1, \$s2, 100</code>  |
| sll  | R      | 0       | 0  | 18 | 17  | 10 | 0  | sll <code>\$s1, \$s2, 10</code>   |
| srl  | R      | 0       | 0  |    |     |    |    | srl <code>\$s1, \$s2, 10</code>   |
| beq  | I      | 4       | 17 | 18 | 25  |    |    | beq <code>\$s1, \$s2, 100</code>  |



98 of 689

## 2.7

## Supporting Procedures in Computer Hardware

A **procedure** or function is one tool C or Java programmers use to structure programs, both to make them easier to understand and to allow code to be reused. Procedures allow the programmer to concentrate on just one portion of the task at a time, with parameters acting as a barrier between the procedure and the rest of the program and data, allowing it to be passed values and return results. We describe the equivalent in Java at the end of this section, but Java needs everything from a computer that C needs.

You can think of a procedure like a spy who leaves with a secret plan, acquires resources, performs the task, covers his tracks, and then returns to the point of origin with the desired result. Nothing else should be perturbed once the mission is complete. Moreover, a spy operates on only a “need to know” basis, so the spy can’t make assumptions about his employer.

Similarly, in the execution of a procedure, the program must follow these six steps:

1. Place parameters in a place where the procedure can access them.
2. Transfer control to the procedure.
3. Acquire the storage resources needed for the procedure.
4. Perform the desired task.
5. Place the result value in a place where the calling program can access it.
6. Return control to the point of origin, since a procedure can be called from several points in a program.

As mentioned above, registers are the fastest place to hold data in a computer, so we want to use them as much as possible. MIPS software follows the following convention in allocating its 32 registers for procedure calling:

- `$a0–$a3`: four argument registers in which to pass parameters
- `$v0–$v1`: two value registers in which to return values
- `$ra`: one return address register to return to the point of origin

In addition to allocating these registers, MIPS assembly language includes an instruction just for the procedures: it jumps to an address and simultaneously saves the address of the following instruction in register `$ra`. The **jump-and-link instruction** (`jal`) is simply written

**procedure** A stored subroutine that performs a specific task based on the parameters with which it is provided.

**jump-and-link instruction** An instruction that jumps to an address and simultaneously saves the address of the following instruction in a register (`$ra` in MIPS).

```
jal ProcedureAddress
```

The *link* portion of the name means that an address or link is formed that points to the calling site to allow the procedure to return to the proper address. This “link,” stored in register `$ra`, is called the **return address**. The return address is needed because the same procedure could be called from several parts of the program.

Implicit in the stored-program idea is the need to have a register to hold the address of the current instruction being executed. For historical reasons, this register is almost always called the **program counter**, abbreviated *PC* in the MIPS architecture, although a more sensible name would have been *instruction address register*. The `jal` instruction saves `PC + 4` in register `$ra` to link to the following instruction to set up the procedure return.

To support such situations, computers like MIPS use a *jump register* instruction (`jr`), meaning an unconditional jump to the address specified in a register:

```
jr $ra
```

The jump register instruction jumps to the address stored in register `$ra`—which is just what we want. Thus, the calling program, or **caller**, puts the parameter values in `$a0–$a3` and uses `jal X` to jump to procedure `X` (sometimes named the **callee**). The callee then performs the calculations, places the results in `$v0–$v1`, and returns control to the caller using `jr $ra`.

### Using More Registers

Suppose a compiler needs more registers for a procedure than the four argument and two return value registers. Since we must cover our tracks after our mission is complete, any registers needed by the caller must be restored to the values that

**return address** A link to the calling site that allows a procedure to return to the proper address; in MIPS it is stored in register `$ra`.

**program counter (PC)** The register containing the address of the instruction in the program being executed

**caller** The program that instigates a procedure and provides the necessary parameter values.

**callee** A procedure that executes a series of stored instructions based on parameters provided by the caller and then returns control to the caller.

**addressing mode** One of several addressing regimes delimited by their varied use of operands and/or addresses.

### MIPS Addressing Mode Summary

Multiple forms of addressing are generically called **addressing modes**. The MIPS addressing modes are the following:

1. **Register addressing**, where the operand is a register
2. **Base or displacement addressing**, where the operand is at the memory location whose address is the sum of a register and a constant in the instruction
3. **Immediate addressing**, where the operand is a constant within the instruction itself
4. **PC-relative addressing**, where the address is the sum of the PC and a constant in the instruction
5. **Pseudodirect addressing**, where the jump address is the 26 bits of the instruction concatenated with the upper bits of the PC

### Hardware Software Interface

Although we show the MIPS architecture as having 32-bit addresses, nearly all microprocessors (including MIPS) have 64-bit address extensions (see [Appendix D](#)). These extensions were in response to the needs of software for larger programs. The process of instruction set extension allows architectures to expand in a way that lets software move compatibly upward to the next generation of architecture.

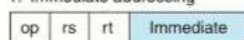
Note that a single operation can use more than one addressing mode. Add, for example, uses both immediate (`addi`) and register (`add`) addressing. Figure 2.24 shows how operands are identified for each addressing mode. [In More Depth](#) shows other addressing modes found in the IBM PowerPC.

### Decoding Machine Language

Sometimes you are forced to reverse-engineer machine language to create the original assembly language. One example is when looking at a core dump. Figure 2.25 shows the MIPS encoding of the fields for the MIPS machine language. This figure helps when translating by hand between assembly language and machine language.

## 2.9 MIPS Addressing for 32-Bit Immediates and Addresses

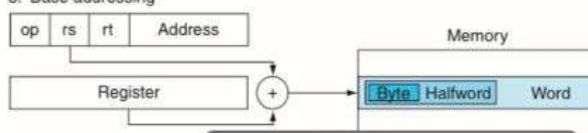
### 1. Immediate addressing



### 2. Register addressing



### 3. Base addressing



Done

patterson\_book-CA.pdf

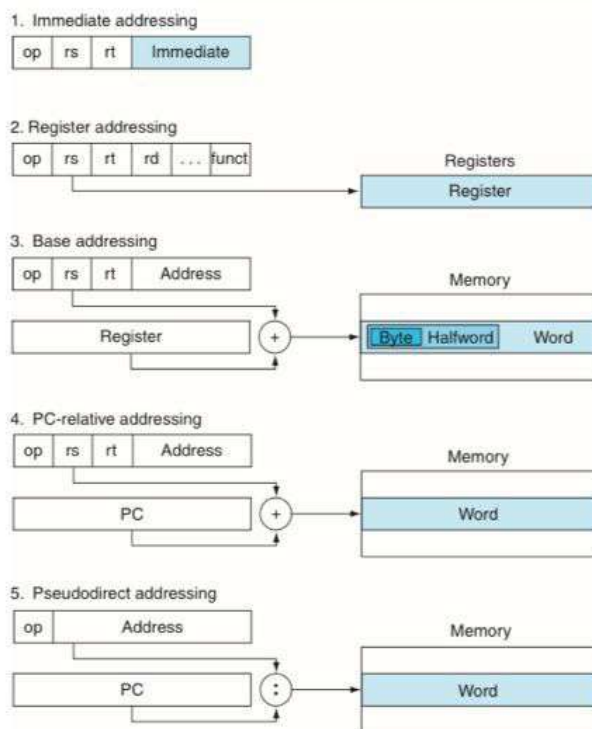


shows other addressing modes found in the IBM PowerPC.

### Addressing Machine Language

120 of 689

times you are forced to reverse-engineer machine language to create the original assembly language. One example is when looking at a core dump. Figure 2.25 shows the MIPS encoding of the fields for the MIPS machine language. This figure helps when translating by hand between assembly language and machine language.



**FIGURE 2.24 Illustration of the five MIPS addressing modes.** The operands are shaded in color. The operand of mode 3 is in memory, whereas the operand for mode 2 is a register. Note that versions of load and store access bytes, halfwords, or words. For mode 1, the operand is 16 bits of the instruction itself. Modes 4 and 5 address instructions in memory, with mode 4 adding a 16-bit address shifted left 2 bits to the PC and mode 5 concatenating a 26-bit address shifted left 2 bits with the 4 upper bits of the PC.



Done

patterson\_book-CA.pdf



121 of 689

## EXAMPLE

## Decoding Machine Code

What is the assembly language statement corresponding to this machine instruction?

```
00af8020hex
```

## ANSWER

The first step in converting hexadecimal to binary is to find the op fields:

```
(Bits: 31 28 26                    5 2 0)
0000 0000 1010 1111 1000 0000 0010 0000
```

We look at the op field to determine the operation. Referring to Figure 2.25, when bits 31–29 are 000 and bits 28–26 are 000, it is an R-format instruction. Let's reformat the binary instruction into R-format fields, listed in Figure 2.26:

```
op      rs      rt      rd      shamt    funct
000000  00101  01111  10000  00000    100000
```

The bottom portion of Figure 2.25 determines the operation of an R-format instruction. In this case, bits 5–3 are 100 and bits 2–0 are 000, which means this binary pattern represents an `add` instruction.

We decode the rest of the instruction by looking at the field values. The decimal values are 5 for the rs field, 15 for rt, 16 for rd (shamt is unused). Figure 2.18 says these numbers represent registers \$a1, \$t7, and \$s0. Now we can show the assembly instruction:

```
add $s0,$a1,$t7
```

Figure 2.26 shows all the MIPS instruction formats. Figure 2.27 shows the MIPS assembly language revealed in Chapter 2; the remaining hidden portion of MIPS instructions deals mainly with arithmetic covered in the next chapter.

|       | op(31:26) |               |            |                    |             |           |           |                |
|-------|-----------|---------------|------------|--------------------|-------------|-----------|-----------|----------------|
| 28–26 | 0(000)    | 1(001)        | 2(010)     | 3(011)             | 4(100)      | 5(101)    | 6(110)    | 7(111)         |
| 31–29 | 0(000)    | R-format      | Bltz/gez   | Jump               | Jump & link | branch eq | branch ne | blez           |
|       |           |               |            |                    |             |           |           | bgtz           |
|       | 1(001)    | add immediate | addiu      | set less than imm. | slliu       | andi      | ori       | xori           |
|       |           |               |            |                    |             |           |           | load upper imm |
|       | 2(010)    | TLB           | FIPt       |                    |             |           |           |                |
|       | 3(011)    |               |            |                    |             |           |           |                |
|       | 4(100)    | load byte     | load half  | lwl                | load word   | ldu       | ldu       | lwr            |
|       |           |               |            |                    |             |           |           |                |
|       | 5(101)    | store byte    | store half | swl                | store word  |           |           | swr            |
|       |           |               |            |                    |             |           |           |                |
|       | 6(110)    | lwd           | lwd        |                    |             |           |           |                |

Done

patterson\_book-CA.pdf



representation.

050 + 01 = with binary or hexadecimal notation.

Si  
Ti 190 of 689No. ...  
Athletes at a Football Factory,  
David Letterman et al., *Book of  
Top Ten Lists*, 1990

## 3.3

## Addition and Subtraction

Addition is just what you would expect in computers. Digits are added bit by bit from right to left, with carries passed to the next digit to the left, just as you would do by hand. Subtraction uses addition: The appropriate operand is simply negated before being added.

## EXAMPLE

## Binary Addition and Subtraction

Let's try adding  $6_{\text{ten}}$  to  $7_{\text{ten}}$  in binary and then subtracting  $6_{\text{ten}}$  from  $7_{\text{ten}}$  in binary.

## 3.3 Addition and Subtraction

171

$$\begin{array}{r}
 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111_{\text{two}} = 7_{\text{ten}} \\
 + \quad 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0110_{\text{two}} = 6_{\text{ten}} \\
 \hline
 = \quad 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1101_{\text{two}} = 13_{\text{ten}}
 \end{array}$$

ANSWER

The 4 bits to the right have all the action; Figure 3.2 shows the sums and carries. The carries are shown in parentheses, with the arrows showing how they are passed.



**FIGURE 3.2** Binary addition, showing carries from right to left. The rightmost bit adds 1 to 0, resulting in the sum of this bit being 1 and the carry out from this bit being 0. Hence, the operation for the second digit to the right is  $0 + 1 + 1$ . This generates a 0 for this sum bit and a carry out of 1. The third digit is the sum of  $1 + 1 + 1$ , resulting in a carry out of 1 and a sum bit of 1. The fourth bit is  $1 + 0 + 0$ , yielding a 1 sum and no carry.

Subtracting  $6_{\text{ten}}$  from  $7_{\text{ten}}$  can be done directly:

$$\begin{array}{r}
 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111_{\text{two}} = 7_{\text{ten}} \\
 - \quad 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0110_{\text{two}} = 6_{\text{ten}} \\
 \hline
 = \quad 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{\text{two}} = 1_{\text{ten}}
 \end{array}$$

or via addition using the two's complement representation of  $-6$ :

$$\begin{array}{r}
 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111_{\text{two}} = 7_{\text{ten}} \\
 + \quad 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1010_{\text{two}} = -6_{\text{ten}} \\
 \hline
 = \quad 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{\text{two}} = 1_{\text{ten}}
 \end{array}$$

We said earlier that overflow occurs when the result from an operation cannot be represented with the available hardware, in this case a 32-bit word. When can overflow occur in addition? When adding operands with different signs, overflow cannot occur. The reason is the sum must be no larger than one of the operands. For example,  $-10 + 4 = -6$ . Since the operands fit in 32 bits and the sum is no larger than an operand, the sum must fit in 32 bits as well. Therefore no overflow can occur when adding positive and negative operands.

There are similar restrictions to the occurrence of overflow during subtract, but it's just the opposite principle: When the signs of the operands are the same, overflow cannot occur. To see this, remember that  $x - y = x + (-y)$  because we subtract by negating the second operand and then add. So, when we subtract operands of

Done

patterson\_book-CA.pdf



|         |          |          |          |
|---------|----------|----------|----------|
| $A + B$ | $< 0$    | $< 0$    | $\geq 0$ |
| $A - B$ | $\geq 0$ | $< 0$    | $< 0$    |
| $A - B$ | $< 0$    | $\geq 0$ | $\geq 0$ |

FIGURE 3.3 Overflow conditions for addition and subtraction.

The computer designer must decide how to handle arithmetic overflows. Although some languages like C ignore integer overflow, languages like Ada and Fortran require that the program be notified. The programmer or the programming environment must then decide what to do when overflow occurs.

MIPS detects overflow with an **exception**, also called an **interrupt** on many computers. An exception or interrupt is essentially an unscheduled procedure call. The address of the instruction that overflowed is saved in a register, and the computer jumps to a predefined address to invoke the appropriate routine for that exception. The interrupted address is saved so that in some situations the program can continue after corrective code is executed. (Section 5.6 covers exceptions in more detail; Chapters 7 and 8 describe other situations where exceptions and interrupts occur.)

MIPS includes a register called the *exception program counter* (EPC) to contain the address of the instruction that caused the exception. The instruction *move from system control* (`mfc0`) is used to copy EPC into a general-purpose register so that MIPS software has the option of returning to the offending instruction via a jump register instruction.

### Hardware Software Interface

**exception** Also called interrupt. An unscheduled event that disrupts program execution; used to detect overflow.

**interrupt** An exception that comes from outside of the processor. (Some architectures use the term *interrupt* for all exceptions.)

**Elaboration:** MIPS can trap on overflow, but unlike many other computers there is no conditional branch to test overflow. A sequence of MIPS instructions can discover overflow. For signed addition, the sequence is the following (see the In More Depth segment on logical instruction in Chapter 2 for the definition of the `xor` instructions):

```
addu $t0, $t1, $t2 # $t0 = sum, but don't trap
xor  $t3, $t1, $t2 # Check if signs differ
slt  $t3, $t3, $zero # $t3 = 1 if signs differ
bne  $t3, $zero, No_overflow # $t1, $t2 signs ≠,
so no overflow
xor  $t3, $t0, $t1 # signs =; sign of sum match too?
      # $t3 negative if sum sign different
slt  $t3, $t3, $zero # $t3 = 1 if sum sign different
bne  $t3, $zero, Overflow # All three signs ≠; go to
overflow
```





Done

patterson\_book-CA.pdf



195 of 689

6

Chapter 3 Arithmetic for Computers

*ultiplication is vexation,  
ivision is as bad;  
he rule of three doth puzzle  
e,  
ad practice drives me mad.*

onymous, Elizabethan  
anuscript, 1570

## 3.4

## Multiplication

Now that we have completed the explanation of addition and subtraction, we are ready to build the more vexing operation of multiply.

But first let's review the multiplication of decimal numbers in longhand to remind ourselves of the steps and the names of the operands. For reasons that will become clear shortly, we limit this decimal example to using only the digits 0 and 1. Multiplying  $1000_{\text{ten}}$  by  $1001_{\text{ten}}$ :

|              |          |                        |
|--------------|----------|------------------------|
| Multiplicand |          | $1000_{\text{ten}}$    |
| Multiplier   | $\times$ | $1001_{\text{ten}}$    |
|              |          | <hr/>                  |
|              |          | 1000                   |
|              |          | 0000                   |
|              |          | 0000                   |
|              |          | 1000                   |
|              |          | <hr/>                  |
| Product      |          | $1001000_{\text{ten}}$ |

The first operand is called the *multiplicand* and the second the *multiplier*. The final result is called the *product*. As you may recall, the algorithm learned in grammar school is to take the digits of the multiplier one at a time from right to left, multiplying the multiplicand by the single digit of the multiplier and shifting the intermediate product one digit to the left of the earlier intermediate products.

The first observation is that the number of digits in the product is considerably larger than the number in either the multiplicand or the multiplier. In fact, if we ignore the sign bits, the length of the multiplication of an  $n$ -bit multiplicand and an  $m$ -bit multiplier is a product that is  $n + m$  bits long. That is,  $n + m$  bits are required to represent all possible products. Hence, like add, multiply must cope with overflow because we frequently want a 32-bit product as the result of multiplying two 32-bit numbers.

In this example we restricted the decimal digits to 0 and 1. With only two choices, each step of the multiplication is simple:

1. Just place a copy of the multiplicand ( $1 \times$  multiplicand) in the proper place if the multiplier digit is a 1, or
2. Place 0 ( $0 \times$  multiplicand) in the proper place if the digit is 0.

Although the decimal example above happened to use only 0 and 1, multiplication of binary numbers must always use 0 and 1, and thus always offers only these two choices.

Now that we have reviewed the basics of multiplication, the traditional next step is to provide the highly optimized multiply hardware. We break with tradition in the belief that you will gain a better understanding by seeing the evolution of the multiply hardware and algorithm through multiple generations. For now, let's assume that we are multiplying only positive numbers.

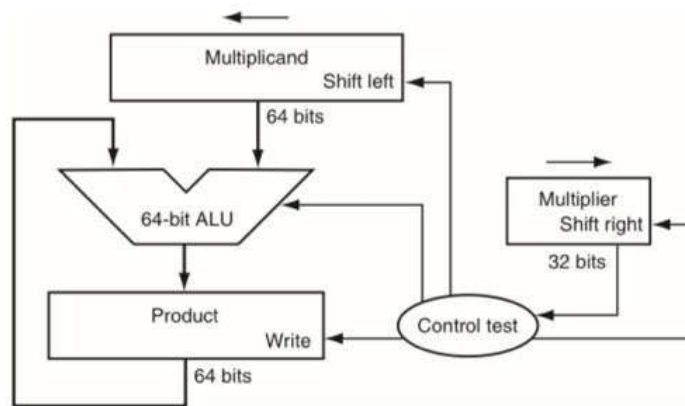
Although the decimal example above happened to use only 0 and 1, multiplication of binary numbers must always use 0 and 1, and thus always offers only these two choices.

Now that we have reviewed the basics of multiplication, the traditional next step is to provide the highly optimized multiply hardware. We break with tradition in the belief that you will gain a better understanding by seeing the evolution of the multiply hardware and algorithm through multiple generations. For now, let's assume that we are multiplying only positive numbers.

### Sequential Version of the Multiplication Algorithm and Hardware

This design mimics the algorithm we learned in grammar school; the hardware is shown in Figure 3.5. We have drawn the hardware so that data flows from top to bottom to more closely resemble the paper-and-pencil method.

Let's assume that the multiplier is in the 32-bit Multiplier register and that the 64-bit Product register is initialized to 0. From the paper-and-pencil example above, it's clear that we will need to move the multiplicand left one digit each step as it may be added to the intermediate products. Over 32 steps a 32-bit multiplicand would move 32 bits to the left. Hence we need a 64-bit Multiplicand register, initialized with the 32-bit multiplicand in the right half and 0 in the left half. This register is then shifted left 1 bit each step to align the multiplicand with the sum being accumulated in the 64-bit Product register.



**FIGURE 3.5 First version of the multiplication hardware.** The Multiplicand register, ALU, and Product register are all 64 bits wide, with only the Multiplier register containing 32 bits. The 32-bit multiplicand starts in the right half of the Multiplicand register and is shifted left 1 bit on each step. The multiplier is shifted in the opposite direction at each step. The algorithm starts with the product initialized to 0. Control decides when to shift the Multiplicand and Multiplier registers and when to write new values into the Product register.

Figure 3.6 shows the three basic steps needed for each bit. The least bit of the multiplier (Multiplier0) determines whether the multiplicand is added to the Product register. The left shift in step 2 has the effect of moving the intermediate operands to the left, just as when multiplying by hand. The step 3 gives us the next bit of the multiplier to examine in the following step. These three steps are repeated 32 times to obtain the product. If each

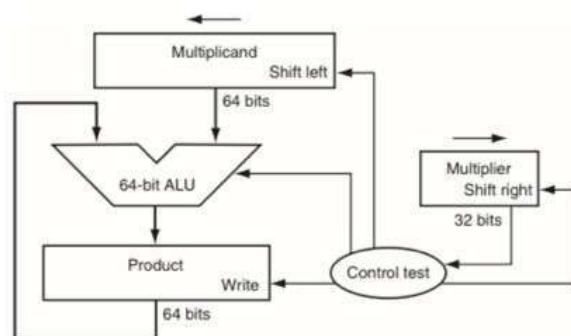
Although the decimal example above happened to use only 0 and 1, multiplication of binary numbers must always use 0 and 1, and thus always offers only these two choices.

Now that we have reviewed the basics of multiplication, the traditional next step is to provide the highly optimized multiply hardware. We break with tradition in the belief that you will gain a better understanding by seeing the evolution of the multiply hardware and algorithm through multiple generations. For now, let's assume that we are multiplying only positive numbers.

### Sequential Version of the Multiplication Algorithm and Hardware

This design mimics the algorithm we learned in grammar school; the hardware is shown in Figure 3.5. We have drawn the hardware so that data flows from top to bottom to more closely resemble the paper-and-pencil method.

Let's assume that the multiplier is in the 32-bit Multiplier register and that the 64-bit Product register is initialized to 0. From the paper-and-pencil example above, it's clear that we will need to move the multiplicand left one digit each step as it may be added to the intermediate products. Over 32 steps a 32-bit multiplicand would move 32 bits to the left. Hence we need a 64-bit Multiplicand register, initialized with the 32-bit multiplicand in the right half and 0 in the left half. This register is then shifted left 1 bit each step to align the multiplicand with the sum being accumulated in the 64-bit Product register.



**FIGURE 3.5 First version of the multiplication hardware.** The Multiplicand register, ALU, and Product register are all 64 bits wide, with only the Multiplier register containing 32 bits. The 32-bit multiplicand starts in the right half of the Multiplicand register and is shifted left 1 bit on each step. The multiplier is shifted in the opposite direction at each step. The algorithm starts with the product initialized to 0. Control decides when to shift the Multiplicand and Multiplier registers and when to write new values into the Product register.

Figure 3.6 shows the three basic steps needed for each bit. The least significant bit of the multiplier (Multiplier0) determines whether the multiplicand is added to the Product register. The left shift in step 2 has the effect of moving the intermediate operands to the left, just as when multiplying by hand. The shift right in step 3 gives us the next bit of the multiplier to examine in the following iteration. These three steps are repeated 32 times to obtain the product. If each step took a



Done

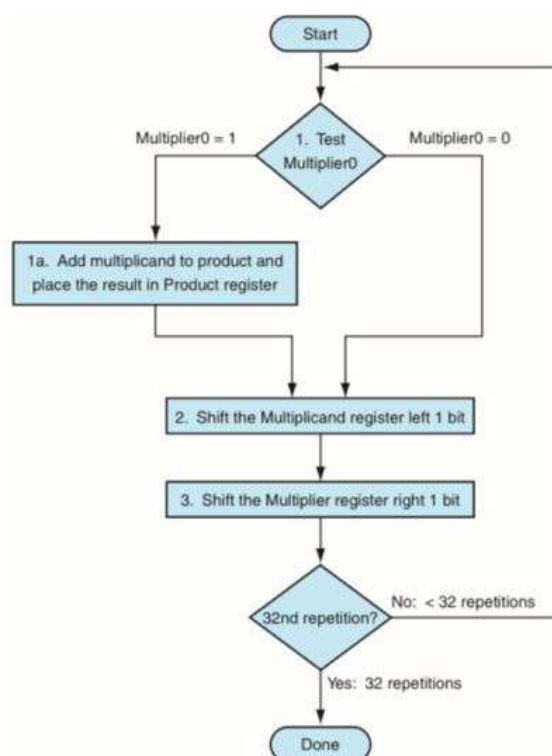
patterson\_book-CA.pdf



197 of 689

**the multiplication hardware.** The Multiplicand register, ALU, and with only the Multiplier register containing 32 bits. The 32-bit multiplier Multiplicand register and is shifted left 1 bit on each step. The multiplication at each step. The algorithm starts with the product initialized to 0. Control decides when to shift the Multiplicand and Multiplier registers and when to write new values into the Product register.

Figure 3.6 shows the three basic steps needed for each bit. The least significant bit of the multiplier (Multiplier0) determines whether the multiplicand is added to the Product register. The left shift in step 2 has the effect of moving the intermediate operands to the left, just as when multiplying by hand. The shift right in step 3 gives us the next bit of the multiplier to examine in the following iteration. These three steps are repeated 32 times to obtain the product. If each step took a



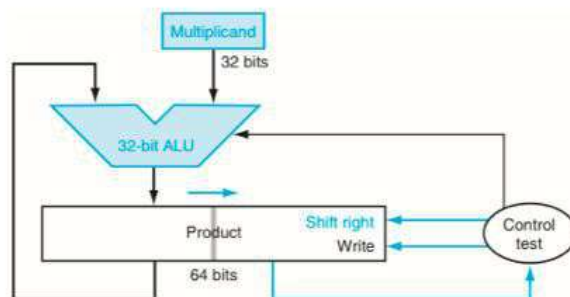
**FIGURE 3.6** The first multiplication algorithm, using the hardware shown in Figure 3.5. If the least significant bit of the multiplier is 1, add the multiplicand to the product. If not, go to the next step. Shift the multiplicand left and the multiplier right in the next two steps. These three steps are repeated 32 times.



Replacing arithmetic by shifts can also occur when multiplying by constants. Some compilers replace multiplies by short constants with a series of shifts and left represents a number twice as large in base 2, shift-effect as multiplying by a power of 2. As mentioned compiler will perform the strength reduction optimization of substituting a left shift for a multiply by a power of 2.

**Hardware  
Software  
Interface**

199 of 689



**FIGURE 3.7 Refined version of the multiplication hardware.** Compare with the first version in Figure 3.5. The Multiplicand register, ALU, and Multiplier register are all 32 bits wide, with only the Product register left at 64 bits. Now the product is shifted right. The separate Multiplier register also disappeared. The multiplier is placed instead in the right half of the Product register. These changes are highlighted in color.

### EXAMPLE

### ANSWER

#### A Multiply Algorithm

Using 4-bit numbers to save space, multiply  $2_{\text{ten}} \times 3_{\text{ten}}$ , or  $0010_{\text{two}} \times 0011_{\text{two}}$ .

Figure 3.8 shows the value of each register for each of the steps labeled according to Figure 3.6, with the final value of  $0000\ 0110_{\text{two}}$  or  $6_{\text{ten}}$ . Color is used to indicate the register values that change on that step, and the bit circled is the one examined to determine the operation of the next step.

#### Signed Multiplication

So far we have dealt with positive numbers. The easiest way to understand how to deal with signed numbers is to first convert the multiplier and multiplicand to positive numbers and then remember the original signs. The algorithms should then be run for 31 iterations, leaving the signs out of the calculation. As we learned in grammar school, we need negate the product only if the original signs disagree.

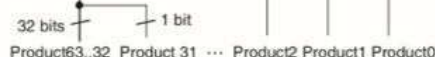
It turns out that the last algorithm will work for signed numbers provided that we remember that the numbers we are dealing with have infinite digits, and that we are only representing them with 32 bits. Hence, the shifting steps would need to extend the sign of the product for signed numbers. When the algorithm completes, the lower word would have the 32-bit product.

| Iteration | Step                                                         | Multiplier | Multiplicand | Product   |
|-----------|--------------------------------------------------------------|------------|--------------|-----------|
| 0         | Initial values                                               | 0011       | 0000 0010    | 0000 0000 |
| 1         | 1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mrand}$ | 0011       | 0000 0010    | 0000 0010 |
|           | 2: Shift left Multiplier                                     | 0011       | 0000 0100    | 0000 0010 |
|           | 3: Shift right Multiplier                                    | 0001       | 0000 0100    | 0000 0010 |
| 2         | 1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mrand}$ | 0001       | 0000 0100    | 0000 0110 |
|           | 2: Shift left Multiplier                                     | 0001       | 0000 1000    | 0000 0110 |
|           | 3: Shift right Multiplier                                    | 0000       | 0000 1000    | 0000 0110 |
| 3         | 1: $0 \Rightarrow$ no operation                              | 0000       | 0000 1000    | 0000 0110 |
|           | 2: Shift left Multiplier                                     | 0000       | 0001 0000    | 0000 0110 |
|           | 3: Shift right Multiplier                                    | 0000       | 0001 0000    | 0000 0110 |
| 4         | 1: $0 \Rightarrow$ no operation                              | 0000       | 0001 0000    | 0000 0110 |
|           | 2: Shift left Multiplier                                     | 0000       | 0010 0000    | 0000 0110 |
|           | 3: Shift right Multiplier                                    | 0000       | 0010 0000    | 0000 0110 |

**FIGURE 3.8 Multiply example using algorithm in Figure 3.6.** The bit examined to determine the next step is circled in color.

patterson\_book-CA.pdf

202 of 689



**FIGURE 3.9 Fast multiplication hardware.** Rather than use a single 32-bit adder 32 times, this hardware “unrolls the loop” to use 32 adders. Each adder produces a 32-bit sum and a carry out. The least significant bit is a bit of the product, and the carry out and the upper 31 bits of the sum are passed along to the next adder.

### 3.5 Division

*Divide et impera.*

Latin for “Divide and rule,” ancient political maxim cited by Machiavelli, 1532.

The reciprocal operation of multiply is divide, an operation that is even less frequent and even more quirky. It even offers the opportunity to perform a mathematically invalid operation: dividing by 0.

Let’s start with an example of long division using decimal numbers to recall the names of the operands and the grammar school division algorithm. For reasons similar to those in the previous section, we limit the decimal digits to just 0 or 1. The example is dividing  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

$$\begin{array}{r}
 \text{Quotient} \\
 1001_{\text{ten}} \\
 \text{Divisor } 1000_{\text{ten}} \overline{) 1001010_{\text{ten}}} \\
 \underline{-1000} \\
 10 \\
 101 \\
 1010 \\
 \underline{-1000} \\
 10_{\text{ten}} \\
 \text{Remainder}
 \end{array}$$

The two operands (**dividend** and **divisor**) and the result (**quotient**) of divide are accompanied by a second result called the **remainder**. Here is another way to express the relationship between the components:

$$\text{Dividend} = \text{Quotient} \times \text{Divisor} + \text{Remainder}$$

where the remainder is smaller than the divisor. Infrequently, programs use the divide instruction just to get the remainder, ignoring the quotient.

The basic grammar school division algorithm tries to see how big a number can be subtracted, creating a digit of the quotient on each attempt. Our carefully selected decimal example uses only the numbers 0 and 1, so it’s easy to figure out how many times the divisor goes into the portion of the dividend: it’s either 0 times or 1 time. Binary numbers contain only 0 or 1, so binary division is restricted to these two choices, thereby simplifying binary division.

Let’s assume that both the dividend and divisor are positive and hence the quotient and the remainder are nonnegative. The division operands and both results are 32-bit values, and we will ignore the sign for now.

**dividend** A number being divided.

**divisor** A number that the dividend is divided by.

**quotient** The primary result of a division; a number that when multiplied by the divisor and added to the remainder produces the dividend.

**remainder** The secondary result of a division; a number that when added to the product of the quotient and the divisor produces the dividend.

### A Division Algorithm and Hardware

Figure 3.10 shows how the grammar school algorithm. We start with the 32-bit Quotient register set to 0. Each iteration of the algorithm needs to



Done

patterson\_book-CA.pdf



how many times the divisor goes into the portion of the dividend: it's either 0 times or 1 time. Binary numbers contain only 0 or 1, so binary division is simpler, thereby simplifying binary division.

203 of 689

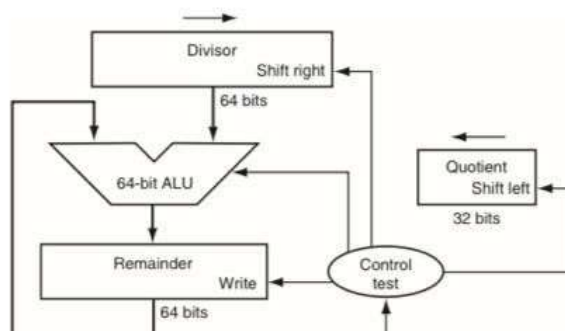
The dividend and divisor are positive and hence the quotient is nonnegative. The division operands and both results are 32-bit values, and we will ignore the sign for now.

result of a division; a number that when added to the product of the quotient and the divisor produces the dividend.

### A Division Algorithm and Hardware

Figure 3.10 shows hardware to mimic our grammar school algorithm. We start with the 32-bit Quotient register set to 0. Each iteration of the algorithm needs to move the divisor to the right one digit, so we start with the divisor placed in the left half of the 64-bit Divisor register and shift it right 1 bit each step to align it with the dividend. The Remainder register is initialized with the dividend.

Figure 3.11 shows three steps of the first division algorithm. Unlike a human, the computer isn't smart enough to know in advance whether the divisor is smaller than the dividend. It must first subtract the divisor in step 1; remember that this is how we performed the comparison in the set on less than instruction. If the result is positive, the divisor was smaller or equal to the dividend, so we generate a 1 in the quotient (step 2a). If the result is negative, the next step is to restore the original value by adding the divisor back to the remainder and generate a 0 in the quotient (step 2b). The divisor is shifted right and then we iterate again. The remainder and quotient will be found in their namesake registers after the iterations are complete.



**FIGURE 3.10 First version of the division hardware.** The Divisor register, ALU, and Remainder register are all 64 bits wide, with only the Quotient register being 32 bits. The 32-bit divisor starts in the left half of the Divisor register and is shifted right 1 bit on each iteration. The remainder is initialized with the dividend. Control decides when to shift the Divisor and Quotient registers and when to write the new value into the Remainder register.

Done

patterson\_book-CA.pdf



the remainder is negative, it simply adds the dividend to the shifted remainder in the following step since  $(r + d) \times 2 - d = r \times 2 + d \times 2 - d = r \times 2 + d$ . This *nonrestoring* division, which takes two clock cycles per step, is explored further in Exercise 3.29; the restoring division.

209 of 689

## 3.6

## Floating Point

Speed gets you nowhere if you're headed the wrong way.

American proverb

Going beyond signed and unsigned integers, programming languages support numbers with fractions, which are called *reals* in mathematics. Here are some examples of reals:

$3.14159265 \dots_{\text{ten}}$  ( $\pi$ )

$2.71828 \dots_{\text{ten}}$  ( $e$ )

$0.000000001_{\text{ten}}$  or  $1.0_{\text{ten}} \times 10^{-9}$  (seconds in a nanosecond)

$3,155,760,000_{\text{ten}}$  or  $3.15576_{\text{ten}} \times 10^9$  (seconds in a typical century)

Notice that in the last case, the number didn't represent a small fraction, but it was bigger than we could represent with a 32-bit signed integer. The alternative notation for the last two numbers is called **scientific notation**, which has a single digit to the left of the decimal point. A number in scientific notation that has no leading 0s is called a **normalized** number, which is the usual way to write it. For example,  $1.0_{\text{ten}} \times 10^{-9}$  is in normalized scientific notation, but  $0.1_{\text{ten}} \times 10^{-8}$  and  $10.0_{\text{ten}} \times 10^{-10}$  are not.

**scientific notation** A notation that renders numbers with a single digit to the left of the decimal point.

**normalized** A number in floating-point notation that has no leading 0s.

MIPS assembly language

| Category           | Instruction                    | Example             | Meaning                                               | Comments                             |
|--------------------|--------------------------------|---------------------|-------------------------------------------------------|--------------------------------------|
| Arithmetic         | add                            | add \$s1,\$s2,\$s3  | $\$s1 = \$s2 + \$s3$                                  | Three operands; overflow detected    |
|                    | subtract                       | sub \$s1,\$s2,\$s3  | $\$s1 = \$s2 - \$s3$                                  | Three operands; overflow detected    |
|                    | add immediate                  | addi \$s1,\$s2,100  | $\$s1 = \$s2 + 100$                                   | + constant; overflow detected        |
|                    | add unsigned                   | addu \$s1,\$s2,\$s3 | $\$s1 = \$s2 + \$s3$                                  | Three operands; overflow undetected  |
|                    | subtract unsigned              | subu \$s1,\$s2,\$s3 | $\$s1 = \$s2 - \$s3$                                  | Three operands; overflow undetected  |
|                    | add immediate unsigned         | addiu \$s1,\$s2,100 | $\$s1 = \$s2 + 100$                                   | + constant; overflow undetected      |
|                    | move from coprocessor register | mfc0 \$s1,\$epc     | $\$s1 = \$epc$                                        | Copy Exception PC + special regs     |
|                    | multiply                       | mult \$s2,\$s3      | $Hi, Lo = \$s2 \times \$s3$                           | 64-bit signed product in Hi, Lo      |
|                    | multiply unsigned              | multu \$s2,\$s3     | $Hi, Lo = \$s2 \times \$s3$                           | 64-bit unsigned product in Hi, Lo    |
|                    | divide                         | div \$s2,\$s3       | $Lo = \$s2 / \$s3$ ,<br>$Hi = \$s2 \text{ mod } \$s3$ | Lo = quotient, Hi = remainder        |
| Data transfer      | divide unsigned                | divu \$s2,\$s3      | $Lo = \$s2 / \$s3$ ,<br>$Hi = \$s2 \text{ mod } \$s3$ | Unsigned quotient and remainder      |
|                    | move from Hi                   | mflr \$s1           | $\$s1 = Hi$                                           | Used to get copy of Hi               |
|                    | move from Lo                   | mflr \$s1           | $\$s1 = Lo$                                           | Used to get copy of Lo               |
|                    | load word                      | lw \$s1,100(\$s2)   | $\$s1 = \text{Memory}[\$s2 + 100]$                    | Word from memory to register         |
|                    | store word                     | sw \$s1,100(\$s2)   | $\text{Memory}[\$s2 + 100] = \$s1$                    | Word from register to memory         |
|                    | load half unsigned             | lhu \$s1,100(\$s2)  | $\$s1 = \text{Memory}[\$s2 + 100]$                    | Halfword memory to register          |
|                    | store half                     | sh \$s1,100(\$s2)   | $\text{Memory}[\$s2 + 100] = \$s1$                    | Halfword register to memory          |
|                    | load byte unsigned             | lbu \$s1,100(\$s2)  | $\$s1 = \text{Memory}[\$s2 + 100]$                    | Byte from memory to register         |
|                    | store byte                     | sb \$s1,100(\$s2)   | $\text{Memory}[\$s2 + 100] = \$s1$                    | Byte from register to memory         |
|                    | load upper immediate           | lui \$s1,100        | $\$s1 = 100 \times 2^{16}$                            | Loads constant in upper 16 bits      |
| Logical            | and                            | and \$s1,\$s2,\$s3  | $\$s1 = \$s2 \& \$s3$                                 | Three reg. operands; bit-by-bit AND  |
|                    | or                             | or \$s1,\$s2,\$s3   | $\$s1 = \$s2   \$s3$                                  | Three reg. operands; bit-by-bit OR   |
|                    | nor                            | nor \$s1,\$s2,\$s3  | $\$s1 = \sim (\$s2   \$s3)$                           | Three reg. operands; bit-by-bit NOR  |
|                    | and immediate                  | andi \$s1,\$s2,100  | $\$s1 = \$s2 \& 100$                                  | Bit-by-bit AND with constant         |
|                    | or immediate                   | ori \$s1,\$s2,100   | $\$s1 = \$s2   100$                                   | Bit-by-bit OR with constant          |
|                    | shift left logical             | sll \$s1,\$s2,10    | $\$s1 = \$s2 \ll 10$                                  | Shift left by constant               |
|                    | shift right logical            | srl \$s1,\$s2,10    | $\$s1 = \$s2 \gg 10$                                  | Shift right by constant              |
| Conditional branch | branch on equal                | beq \$s1,\$s2,25    | if ( $\$s1 == \$s2$ ) go to PC + 4 + 100              | Equal test; PC-relative branch       |
|                    | branch on not equal            | bne \$s1,\$s2,25    | if ( $\$s1 \neq \$s2$ ) go to PC + 4 + 100            | Not equal test; PC-relative          |
|                    | set on less than               | slt \$s1,\$s2,\$s3  | if ( $\$s2 < \$s3$ ) $\$s1 = 1$ ; else $\$s1 = 0$     | Compare less than; two's complement  |
|                    | set less than immediate        | slti \$s1,\$s2,100  | if ( $\$s2 < 100$ ) $\$s1 = 1$ ; else $\$s1 = 0$      | Compare < constant; two's complement |
|                    | set less than unsigned         | sltu \$s1,\$s2,\$s3 | if ( $\$s2 < \$s3$ ) $\$s1 = 1$ ; else $\$s1 = 0$     | Compare less than; natural numbers   |
|                    | set less than immediate        | sltiu \$s1,\$s2,100 | if ( $\$s2 < 100$ ) $\$s1 = 1$ ; else $\$s1 = 0$      | Compare < constant; natural numbers  |

## 210 of 689

Just as we can show decimal numbers in scientific notation, we can also show binary numbers in scientific notation:

$$1.0_{\text{two}} \times 2^{-1}$$

To keep a binary number in normalized form, we need a base that we can increase or decrease by exactly the number of bits the number must be shifted to have one nonzero digit to the left of the decimal point. Only a base of 2 fulfills our need. Since the base is not 10, we also need a new name for decimal point; *binary point* will do fine.

Computer arithmetic that supports such numbers is called **floating point** because it represents numbers in which the binary point is not fixed, as it is for integers. The programming language C uses the name *float* for such numbers. Just as in scientific notation, numbers are represented as a single nonzero digit to the left of the binary point. In binary, the form is

$$1.\text{xxxxxxxx}_{\text{two}} \times 2^{????}$$

(Although the computer represents the exponent in base 2 as well as the rest of the number, to simplify the notation we show the exponent in decimal.)

A standard scientific notation for reals in normalized form offers three advantages. It simplifies exchange of data that includes floating-point numbers; it simplifies the floating-point arithmetic algorithms to know that numbers will always be in this form; and it increases the accuracy of the numbers that can be stored in a word, since the unnecessary leading 0s are replaced by real digits to the right of the binary point.

### Floating-Point Representation

A designer of a floating-point representation must find a compromise between the size of the **fraction** and the size of the **exponent** because a fixed word size means you must take a bit from one to add a bit to the other. This trade-off is between precision and range: Increasing the size of the fraction enhances the precision of the fraction, while increasing the size of the exponent increases the range of numbers that can be represented. As our design guideline from Chapter 2 reminds us, good design demands good compromise.

Floating-point numbers are usually a multiple of the size of a word. The representation of a MIPS floating-point number is shown below, where *s* is the sign of the floating-point number (1 meaning negative), exponent is the value of the 8-bit exponent field (including the sign of the exponent), and fraction is the 23-bit number. This representation is called *sign and magnitude*, since the sign has a separate bit from the rest of the number.

**floating point** Computer arithmetic that represents numbers in which the binary point is not fixed.

**fraction** The value, generally between 0 and 1, placed in the fraction field.

**exponent** In the numerical representation system of floating-point arithmetic, the value that is placed in the exponent field.

|       |    |    |    |    |    |    |    |    |          |    |    |    |    |    |    |    |    |         |    |    |    |   |   |   |   |   |   |          |   |   |   |  |  |  |  |  |  |
|-------|----|----|----|----|----|----|----|----|----------|----|----|----|----|----|----|----|----|---------|----|----|----|---|---|---|---|---|---|----------|---|---|---|--|--|--|--|--|--|
| 31    | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22       | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13      | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3        | 2 | 1 | 0 |  |  |  |  |  |  |
| s     |    |    |    |    |    |    |    |    | exponent |    |    |    |    |    |    |    |    |         |    |    |    |   |   |   |   |   |   | fraction |   |   |   |  |  |  |  |  |  |
| 1 bit |    |    |    |    |    |    |    |    | 8 bits   |    |    |    |    |    |    |    |    | 23 bits |    |    |    |   |   |   |   |   |   |          |   |   |   |  |  |  |  |  |  |

In general, floating-point numbers are generally of the form

$$(-1)^S \times F \times 2^E$$

*F* involves the value in the fraction field and *E* involves the value in the exponent field; the exact relationship to these fields will be spelled out soon. (We will shortly see that MIPS does something slightly more sophisticated.)

These chosen sizes of exponent and fraction give MIPS computer arithmetic an extraordinary range. Fractions almost as small as  $2.0_{\text{ten}} \times 10^{-38}$  and numbers almost as large as  $2.0_{\text{ten}} \times 10^{38}$  can be represented in a computer. Alas, extraordinary differs from infinite, so it is still possible for numbers to be too large. Thus, overflow interrupts can occur in floating-point arithmetic as well as in integer arithmetic. Notice that **overflow** here means that the exponent is too large to be represented in the exponent field.

Floating point offers a new kind of exceptional event as well. Just as programmers will want to know when they have calculated a number that is too large to be represented, they will want to know if the nonzero fraction they are calculating

**overflow (floating-point)** A situation in which a positive exponent becomes too large to fit in the exponent field.



Done

patterson\_book-CA.pdf



211 of 689



In general, floating-point numbers are generally of the form

$$(-1)^S \times F \times 2^E$$

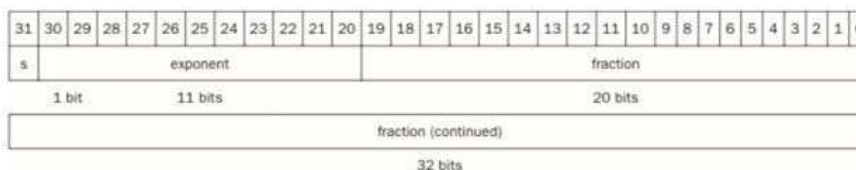
F involves the value in the fraction field and E involves the value in the exponent field; the exact relationship to these fields will be spelled out soon. (We will shortly see that MIPS does something slightly more sophisticated.)

These chosen sizes of exponent and fraction give MIPS computer arithmetic an extraordinary range. Fractions almost as small as  $2.0_{\text{ten}} \times 10^{-38}$  and numbers almost as large as  $2.0_{\text{ten}} \times 10^{38}$  can be represented in a computer. Alas, extraordinary differs from infinite, so it is still possible for numbers to be too large. Thus, overflow interrupts can occur in floating-point arithmetic as well as in integer arithmetic. Notice that **overflow** here means that the exponent is too large to be represented in the exponent field.

Floating point offers a new kind of exceptional event as well. Just as programmers will want to know when they have calculated a number that is too large to be represented, they will want to know if the nonzero fraction they are calculating has become so small that it cannot be represented; either event could result in a program giving incorrect answers. To distinguish it from overflow, people call this event **underflow**. This situation occurs when the negative exponent is too large to fit in the exponent field.

One way to reduce chances of underflow or overflow is to offer another format that has a larger exponent. In C this number is called *double*, and operations on doubles are called **double precision** floating-point arithmetic; **single precision** floating point is the name of the earlier format.

The representation of a double precision floating-point number takes two MIPS words, as shown below, where *s* is still the sign of the number, *exponent* is the value of the 11-bit exponent field, and *fraction* is the 52-bit number in the fraction.



**overflow (floating-point)** A situation in which a positive exponent becomes too large to fit in the exponent field.

**underflow (floating-point)** A situation in which a negative exponent becomes too large to fit in the exponent field.

**double precision** A floating-point value represented in two 32-bit words.

**single precision** A floating-point value represented in a single 32-bit word.

### 3.6 Floating Point

19:

MIPS double precision allows numbers almost as small as  $2.0_{\text{ten}} \times 10^{-308}$  and almost as large as  $2.0_{\text{ten}} \times 10^{308}$ . Although double precision does increase the exponent range, its primary advantage is its greater precision because of the larger significand.

These formats go beyond MIPS. They are part of the *IEEE 754 floating-point standard*, found in virtually every computer invented since 1980. This standard has greatly improved both the ease of porting floating-point programs and the quality of computer arithmetic.

To pack even more bits into the significand, IEEE 754 makes the leading 1 bit of normalized binary numbers implicit. Hence, the number is actually 24 bits long in single precision (implied 1 and a 23-bit fraction), and 53 bits long in double precision (1 + 52). To be precise, we use the term *significand* to represent the 24- or 53-bit number that is 1 plus the fraction, and *fraction* when we mean the 23- or 52-bit number. Since 0 has no leading 1, it is given the reserved exponent value 0 so that the hardware won't attach a leading 1 to it.

Thus  $00 \dots 00_{\text{two}}$  represents 0; the representation of the rest of the numbers uses the form from before with the hidden 1 added:

$$(-1)^S \times (1 + \text{Fraction}) \times 2^E$$

### 3.6 Floating Point

195

Thus IEEE 754 notation can be processed by integer compares to accelerate sorting of floating-point numbers. Let's show the representation.

## Floating-Point Representation

Show the IEEE 754 binary representation of the number  $-0.75_{10}$  in single and double precision.

### EXAMPLE

The number  $-0.75_{\text{ten}}$  is also

$$-3/4_{\text{ten}} \text{ or } -3/2^2_{\text{ten}}$$

It is also represented by the binary fraction

$$-11_{\text{two}}/2^2_{\text{ten}} \text{ or } -0,11_{\text{two}}$$

In scientific notation, the value is

$$-0.11_{\text{two}} \times 2^0$$

and in normalized scientific notation, it is

$$-1.1_{\text{two}} \times 2^{-1}$$

The general representation for a single precision number is

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - 127)}$$

When we subtract the bias 127 from the exponent of  $-1.1_{\text{two}} \times 2^{-1}$ , the result is

$$(-1)^1 \times (1 + .1000\ 0000\ 0000\ 0000\ 0000\ 0000_{\text{two}}) \times 2^{(126-127)}$$

The single precision binary representation of  $-0.75_{\text{ten}}$  is then

|       |    |    |    |    |    |    |    |    |        |    |    |    |    |    |    |    |         |    |    |    |    |   |   |   |   |   |   |   |   |   |   |  |
|-------|----|----|----|----|----|----|----|----|--------|----|----|----|----|----|----|----|---------|----|----|----|----|---|---|---|---|---|---|---|---|---|---|--|
| 31    | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22     | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14      | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |  |
| 1     | 0  | 1  | 1  | 1  | 1  | 1  | 1  | 0  | 1      | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0       | 0  | 0  | 0  | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |  |
| 1 bit |    |    |    |    |    |    |    |    | 8 bits |    |    |    |    |    |    |    | 23 bits |    |    |    |    |   |   |   |   |   |   |   |   |   |   |  |

The double precision representation is

[illegible]

|                                                                     |    |    |    |    |    |    |    |    |    |    |    |         |    |    |    |    |    |    |    |    |    |   |         |   |   |   |   |   |   |   |   |  |  |  |  |  |  |
|---------------------------------------------------------------------|----|----|----|----|----|----|----|----|----|----|----|---------|----|----|----|----|----|----|----|----|----|---|---------|---|---|---|---|---|---|---|---|--|--|--|--|--|--|
| 31                                                                  | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19      | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8       | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |  |  |  |  |  |  |
| 1                                                                   | 0  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 0  | 1       | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0 | 0       | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |  |  |  |  |  |  |
| 1 bit                                                               |    |    |    |    |    |    |    |    |    |    |    | 11 bits |    |    |    |    |    |    |    |    |    |   | 20 bits |   |   |   |   |   |   |   |   |  |  |  |  |  |  |
| 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 |    |    |    |    |    |    |    |    |    |    |    |         |    |    |    |    |    |    |    |    |    |   |         |   |   |   |   |   |   |   |   |  |  |  |  |  |  |
| 32 bits                                                             |    |    |    |    |    |    |    |    |    |    |    |         |    |    |    |    |    |    |    |    |    |   |         |   |   |   |   |   |   |   |   |  |  |  |  |  |  |

Now let's try going the other direction.

[illegible]

Now let's try going the other direction.

### Converting Binary to Decimal Floating Point

What decimal number is represented by this single precision float?

### EXAMPLE

[illegible]

**ANSWER**

The sign bit is 1, the exponent field contains 129, and the fraction field contains  $1 \times 2^{-2} = 1/4$ , or 0.25. Using the basic equation,

$$\begin{aligned} (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})} &= (-1)^1 \times (1 + 0.25) \times 2^{(129 - 127)} \\ &= -1 \times 1.25 \times 2^2 \\ &= -1.25 \times 4 \\ &= -5.0 \end{aligned}$$

In the next sections we will give the algorithms for floating-point addition and multiplication. At their core, they use the corresponding integer operations on the significands, but extra bookkeeping is necessary to handle the exponents and normalize the result. We first give an intuitive derivation of the algorithms in decimal, and then give a more detailed, binary version in the figures.

### 3.6 Floating Point

197

**Elaboration:** In an attempt to increase range without removing bits from the significand, some computers before the IEEE 754 standard used a base other than 2. For example, the IBM 360 and 370 mainframe computers use base 16. Since changing the IBM exponent by one means shifting the significand by 4 bits, “normalized” base 16 numbers can have up to 3 leading bits of 0! Hence, hexadecimal digits mean that up to 3 bits must be dropped from the significand, which leads to surprising problems in the accuracy of floating-point arithmetic, as noted in Section .

### Floating-Point Addition

Let's add numbers in scientific notation by hand to illustrate the problems in floating-point addition:  $9.999_{\text{ten}} \times 10^1 + 1.610_{\text{ten}} \times 10^{-1}$ . Assume that we can store only four decimal digits of the significand and two decimal digits of the exponent.

Step 1. To be able to add these numbers properly, we must align the decimal point of the number that has the smaller exponent. Hence, we need a form of the smaller number,  $1.610_{\text{ten}} \times 10^{-1}$ , that matches the larger exponent. We obtain this by observing that there are multiple representations of an unnormalized floating-point number in scientific notation:

$$1.610_{\text{ten}} \times 10^{-1} = 0.1610_{\text{ten}} \times 10^0 = 0.01610_{\text{ten}} \times 10^1$$

The number on the right is the version we desire, since its exponent matches the exponent of the larger number,  $9.999_{\text{ten}} \times 10^1$ . Thus the first step shifts the significand of the smaller number to the right until its corrected exponent matches that of the larger number. But we can represent only four decimal digits so, after shifting, the number is really:

$0.016_{ten} \times 10^1$

Step 2. Next comes the addition of the significands:

$$\begin{array}{r} 9.999_{\text{ten}} \\ + 0.016_{\text{ten}} \\ \hline 10.015_{\text{ten}} \end{array}$$

The sum is  $10.015_{\text{tcn}} \times 10^1$ .

Step 3. This sum is not in normalized scientific notation, so we need to adjust it:

$$10.015_{\text{ten}} \times 10^1 = 1.0015_{\text{ten}} \times 10^2$$

Thus, after the addition we may have to shift the sum to put it into normalized form, adjusting the exponent appropriately. This example shows



262 of 689

## ANSWER

**Throughput and Response Time**

Do the following changes to a computer system increase throughput, decrease response time, or both?

1. Replacing the processor in a computer with a faster version
2. Adding additional processors to a system that uses multiple processors for separate tasks—for example, searching the World Wide Web.

Decreasing response time almost always improves throughput. Hence, in case 1, both response time and throughput are improved. In case 2, no one task gets work done faster, so only throughput increases. If, however, the demand for processing in the second case was almost as large as the throughput, the system might force requests to queue up. In this case, increasing the throughput could also improve response time, since it would reduce the waiting time in the queue. Thus, in many real computer systems, changing either execution time or throughput often affects the other.

In discussing the performance of computers, we will be primarily concerned with response time for the first few chapters. (In Chapter 8, on input/output systems, we will discuss throughput-related measures.) To maximize performance, we want to minimize response time or execution time for some task. Thus we can relate performance and execution time for a computer X:

## 4.1 Introduction

243

$$\text{Performance}_X = \frac{1}{\text{Execution time}_X}$$

This means that for two computers X and Y, if the performance of X is greater than the performance of Y, we have

$$\begin{aligned} \text{Performance}_X &> \text{Performance}_Y \\ \frac{1}{\text{Execution time}_X} &> \frac{1}{\text{Execution time}_Y} \\ \text{Execution time}_Y &> \text{Execution time}_X \end{aligned}$$

That is, the execution time on Y is longer than that on X, if X is faster than Y.

In discussing a computer design, we often want to relate the performance of two different computers quantitatively. We will use the phrase “X is  $n$  times faster than Y”—or equivalently “X is  $n$  times as fast as Y”—to mean

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = n$$

If X is  $n$  times faster than Y, then the execution time on Y is  $n$  times longer than it is on X:

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution time}_Y}{\text{Execution time}_X} = n$$

**Relative Performance**

If computer A runs a program in 10 seconds and computer B runs the same program in 15 seconds, how much faster is A than B?

We know that A is  $n$  times faster than B if

$$\frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = n$$

Thus the performance ratio is

$$\frac{15}{10} = 1.5$$

and A is therefore 1.5 times faster than B.

## EXAMPLE

## ANSWER

264 of 689

called **CPU time**. The actual time the CPU spends computing for a specific task.

called *wall-clock time*, *response time*, or *elapsed time*. These terms mean the total time to complete a task, including disk accesses, memory accesses, input/output (I/O) activities, operating system overhead—everything.

Computers are often shared, however, and a processor may work on several programs simultaneously. In such cases, the system may try to optimize throughput rather than attempt to minimize the elapsed time for one program. Hence, we often want to distinguish between the elapsed time and the time that the processor is working on our behalf, **CPU execution time** or simply **CPU time**, which

recognizes this distinction, is the time the CPU spends computing for this task and does not include time spent waiting for I/O or running other programs. (Remember, though, that the response time experienced by the user will be the elapsed time of the program, not the CPU time.) CPU time can be further divided into the CPU time spent in the program, called **user CPU time**, and the CPU time spent in the operating system performing tasks on behalf of the program, called **system CPU time**. Differentiating between system and user CPU time is difficult to do accurately because it is often hard to assign responsibility for operating system activities to one user program rather than another and because of the functionality differences among operating systems.

For consistency, we maintain a distinction between performance based on elapsed time and that based on CPU execution time. We will use the term *system performance* to refer to elapsed time on an unloaded system, and use *CPU performance* to refer to user CPU time. We will focus on CPU performance in this chapter, although our discussions of how to summarize performance can be applied to either elapsed time or to CPU time measurements.

Although as computer users we care about time, when we examine the details of a computer it's convenient to think about performance in other metrics. In particular, computer designers may want to think about a computer by using a measure that relates to how fast the hardware can perform basic functions. Almost all computers are constructed using a clock that runs at a constant rate and determines when events take place in the hardware. These discrete time intervals are called **clock cycles** (or ticks, clock ticks, clock periods, clocks, cycles). Designers refer to the length of a **clock period** both as the time for a complete *clock cycle* (e.g., 0.25 nanoseconds, 0.25 ns, 250 picoseconds, or 250 ps) and as the *clock rate* (e.g., 4 gigahertz, or 4 GHz), which is the inverse of the clock period. In the next section, we will formalize the relationship between the clock cycles of the hardware designer and the seconds of the computer user.

**user CPU time** The CPU time spent in a program itself.

**system CPU time** The CPU time spent in the operating system performing tasks on behalf of the program.

**clock cycle** Also called tick, clock tick, clock period, clock, cycle. The time for one clock period, usually of the processor clock, which runs at a constant rate.

**clock period** The length of each clock cycle.

Different applications are sensitive to different aspects of the performance of a computer system. Many applications, especially those running on servers, depend as much on I/O performance, which, in turn, relies on both hardware and software, and total elapsed time measured by a wall clock is the measurement of interest. In some application environments, the user may care about throughput, response time, or a complex combination of the two (e.g., maximum throughput with a worst-case response time). To improve the performance of a program, one must have a clear definition of what performance metric matters and then proceed to look for the performance bottlenecks by measuring program execution and looking for the likely bottlenecks. In the following chapters, we will describe how to search for bottlenecks and improve performance in various parts of the system.

## Understanding Program Performance

### Check Yourself

- Suppose we know that an application that uses both a desktop client and a remote server is limited by network performance. For the following changes state whether only the throughput improves, both response time and throughput improve, or neither improves.
  - An extra network channel is added between the client and the server, increasing the total network throughput and reducing the delay to obtain network access (since there are now two channels).
  - The networking software is improved, thereby reducing the network communication delay, but not increasing throughput.
  - More memory is added to the computer.
- Computer C's performance is 4 times better than the performance of computer B, which runs a given application in 28 seconds. How long will computer C take to run that application?

## patterson\_book-CA.pdf

puter B, which runs a given application in 28 seconds. How long will computer C take to run that application?

266 of 689

## 4.2

## CPU Performance and Its Factors

Users and designers often examine performance using different metrics. If we could relate these different metrics, we could determine the effect of a design change on the performance as seen by the user. Since we are confining ourselves to CPU performance at this point, the bottom-line performance measure is CPU execution time. A simple formula relates the most basic metrics (clock cycles and clock cycle time) to CPU time:

$$\text{CPU execution time for a program} = \text{CPU clock cycles for a program} \times \text{Clock cycle time}$$

Alternatively, because clock rate and clock cycle time are inverses,

$$\text{CPU execution time for a program} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

This formula makes it clear that the hardware designer can improve performance by reducing either the length of the clock cycle or the number of clock cycles required for a program. As we will see in this chapter and later in Chapters 5, 6, and 7, the designer often faces a trade-off between the number of clock cycles

needed for a program and the length of each cycle. Many techniques that decrease the number of clock cycles also increase the clock cycle time.

## Improving Performance

Our favorite program runs in 10 seconds on computer A, which has a 4 GHz clock. We are trying to help a computer designer build a computer, B, that will run this program in 6 seconds. The designer has determined that a substantial increase in the clock rate is possible, but this increase will affect the rest of the CPU design, causing computer B to require 1.2 times as many clock cycles as computer A for this program. What clock rate should we tell the designer to target?

## EXAMPLE

Let's first find the number of clock cycles required for the program on A:

$$\begin{aligned} \text{CPU time}_A &= \frac{\text{CPU clock cycles}_A}{\text{Clock rate}_A} \\ 10 \text{ seconds} &= \frac{\text{CPU clock cycles}_A}{4 \times 10^9 \frac{\text{cycles}}{\text{second}}} \end{aligned}$$

$$\text{CPU clock cycles}_A = 10 \text{ seconds} \times 4 \times 10^9 \frac{\text{cycles}}{\text{second}} = 40 \times 10^9 \text{ cycles}$$

CPU time for B can be found using this equation:

$$\begin{aligned} \text{CPU time}_B &= \frac{1.2 \times \text{CPU clock cycles}_A}{\text{Clock rate}_B} \\ 6 \text{ seconds} &= \frac{1.2 \times 40 \times 10^9 \text{ cycles}}{\text{Clock rate}_B} \end{aligned}$$

$$\text{Clock rate}_B = \frac{1.2 \times 40 \times 10^9 \text{ cycles}}{6 \text{ seconds}} = \frac{8 \times 10^9 \text{ cycles}}{\text{second}} = 8 \text{ GHz}$$

computer B must therefore have twice the clock rate of A to run the program in 6 seconds.

## ANSWER



### Hardware Software Interface

The equations in our previous examples do not include any reference to the number of instructions needed for the program. However, since the compiler clearly generated instructions to execute, and the computer had to execute the instructions to run the program, the execution time must depend on the number of instructions in a program. One way to think about execution time is that it equals the number of instructions executed multiplied by the average time per instruction. Therefore, the number of clock cycles required for a program can be written as

$$\text{CPU clock cycles} = \text{Instructions for a program} \times \text{Average clock cycles per instruction}$$

**clock cycles per instruction (CPI)** Average number of clock cycles per instruction for a program or program fragment.

The term **clock cycles per instruction**, which is the average number of clock cycles each instruction takes to execute, is often abbreviated as CPI. Since different instructions may take different amounts of time depending on what they do, CPI is an average of all the instructions executed in the program. CPI provides one way of comparing two different implementations of the same instruction set architecture, since the instruction count required for a program will, of course, be the same.

### EXAMPLE

### ANSWER

#### Using the Performance Equation

Suppose we have two implementations of the same instruction set architecture. Computer A has a clock cycle time of 250 ps and a CPI of 2.0 for some program, and computer B has a clock cycle time of 500 ps and a CPI of 1.2 for the same program. Which computer is faster for this program, and by how much?

We know that each computer executes the same number of instructions for the program; let's call this number  $I$ . First, find the number of processor clock cycles for each computer:

$$\text{CPU clock cycles}_A = I \times 2.0$$

$$\text{CPU clock cycles}_B = I \times 1.2$$

Now we can compute the CPU time for each computer:

$$\begin{aligned} \text{CPU time}_A &= \text{CPU clock cycles}_A \times \text{Clock cycle time}_A \\ &= I \times 2.0 \times 250 \text{ ps} = 500 \times I \text{ ps} \end{aligned}$$

Likewise, for B:

$$\text{CPU time}_B = I \times 1.2 \times 500 \text{ ps} = 600 \times I \text{ ps}$$

Clearly, computer A is faster. The amount faster is given by the ratio of the execution times:

$$\frac{\text{CPU performance}_A}{\text{CPU performance}_B} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = \frac{600 \times I \text{ ps}}{500 \times I \text{ ps}} = 1.2$$

We can conclude that computer A is 1.2 times as fast as computer B for this program.

We can now write this basic performance equation in terms of instruction count (the number of instructions executed by the program), CPI, and clock cycle time:

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

or

$$\text{CPU time} = \frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}}$$

## 4.2 CPU Performance and Its Factors

where  $C_i$  is the count of the number of instructions of class  $i$  executed,  $CPI_i$  is the average number of cycles per instruction for that instruction class, and  $n$  is the number of instruction classes. Remember that overall CPI for a program will depend on both the number of cycles for each instruction type and the frequency of each instruction type in the program execution.

As we described in Chapter 1, the performance of a program depends on the algorithm, the language, the compiler, the architecture, and the actual hardware. The following table summarizes how these components affect the factors in the CPU performance equation.

## Understanding Program Performance

| Hardware or software component | Affects what?                      | How?                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Algorithm                      | Instruction count, possibly CPI    | The algorithm determines the number of source program instructions executed and hence the number of processor instructions executed. The algorithm may also affect the CPI, by favoring slower or faster instructions. For example, if the algorithm uses more floating-point operations, it will tend to have a higher CPI.                                                                    |
| Programming language           | Instruction count, CPI             | The programming language certainly affects the instruction count, since statements in the language are translated to processor instructions, which determine instruction count. The language may also affect the CPI because of its features; for example, a language with heavy support for data abstraction (e.g., Java) will require indirect calls, which will use higher-CPI instructions. |
| Compiler                       | Instruction count, CPI             | The efficiency of the compiler affects both the instruction count and average cycles per instruction, since the compiler determines the translation of the source language instructions into computer instructions. The compiler's role can be very complex and affect the CPI in complex ways.                                                                                                 |
| Instruction set architecture   | Instruction count, clock rate, CPI | The instruction set architecture affects all three aspects of CPU performance, since it affects the instructions needed for a function, the cost in cycles of each instruction, and the overall clock rate of the processor.                                                                                                                                                                    |

## EXAMPLE

## Comparing Code Segments

A compiler designer is trying to decide between two code sequences for a particular computer. The hardware designers have supplied the following facts:

|     | CPI for this instruction class |   |   |
|-----|--------------------------------|---|---|
|     | A                              | B | C |
| CPI | 1                              | 2 | 3 |

271 of 689

**EXAMPLE****Comparing Code Segments**

A compiler designer is trying to decide between two code sequences for a particular computer. The hardware designers have supplied the following facts:

|     | CPI for this instruction class |   |   |
|-----|--------------------------------|---|---|
|     | A                              | B | C |
| CPI | 1                              | 2 | 3 |

For a particular high-level-language statement, the compiler writer is considering two code sequences that require the following instruction counts:

| Code sequence | Instruction counts for instruction class |   |   |
|---------------|------------------------------------------|---|---|
|               | A                                        | B | C |
| 1             | 2                                        | 1 | 2 |
| 2             | 4                                        | 1 | 1 |

Which code sequence executes the most instructions? Which will be faster? What is the CPI for each sequence?

**ANSWER**

Sequence 1 executes  $2 + 1 + 2 = 5$  instructions. Sequence 2 executes  $4 + 1 + 1 = 6$  instructions. So sequence 1 executes fewer instructions.

We can use the equation for CPU clock cycles based on instruction count and CPI to find the total number of clock cycles for each sequence:

$$\text{CPU clock cycles} = \sum_{i=1}^n (\text{CPI}_i \times C_i)$$

This yields

$$\text{CPU clock cycles}_1 = (2 \times 1) + (1 \times 2) + (2 \times 3) = 2 + 2 + 6 = 10 \text{ cycles}$$

$$\text{CPU clock cycles}_2 = (4 \times 1) + (1 \times 2) + (1 \times 3) = 4 + 2 + 3 = 9 \text{ cycles}$$

So code sequence 2 is faster, even though it actually executes one extra instruction. Since code sequence 2 takes fewer overall clock cycles but has more instructions, it must have a lower CPI. The CPI values can be computed by

$$\begin{aligned} \text{CPI} &= \frac{\text{CPU clock cycles}}{\text{Instruction count}} \\ \text{CPI}_1 &= \frac{\text{CPU clock cycles}_1}{\text{Instruction count}_1} = \frac{10}{5} = 2 \\ \text{CPI}_2 &= \frac{\text{CPU clock cycles}_2}{\text{Instruction count}_2} = \frac{9}{6} = 1.5 \end{aligned}$$

The above example shows the danger of using only one factor (instruction count) to assess performance. When comparing two computers, you must look at all three components, which combine to form execution time. If some of the factors are identical, like the clock rate in the above example, performance can be determined by comparing all the nonidentical factors. Since CPI varies by



Done

patterson\_book-CA.pdf



286 of 689

Cost/performance fallacies and pitfalls have ensnared many a computer architect, including us. Accordingly, this section suffers no shortage of relevant examples. start with a pitfall that has trapped many designers and reveals an important relationship in computer design.

*Pitfall: Expecting the improvement of one aspect of a computer to increase performance by an amount proportional to the size of the improvement.*

This pitfall has visited designers of both hardware and software. A simple design problem illustrates it well. Suppose a program runs in 100 seconds on a computer, with multiply operations responsible for 80 seconds of this time. How much do I have to improve the speed of multiplication if I want my program to run five times faster?

## 4.5 Fallacies and Pitfalls

267

The execution time of the program after I make the improvement is given by the following simple equation known as **Amdahl's law**:

$$\text{Execution time after improvement} = \left( \frac{\text{Execution time affected by improvement}}{\text{Amount of improvement}} + \text{Execution time unaffected} \right)$$

**Amdahl's law** A rule stating that the performance enhancement possible with a given improvement is limited by the amount that the improved feature is used.

For this problem:

$$\text{Execution time after improvement} = \frac{80 \text{ seconds}}{n} + (100 - 80 \text{ seconds})$$

Since we want the performance to be five times faster, the new execution time should be 20 seconds, giving

$$20 \text{ seconds} = \frac{80 \text{ seconds}}{n} + 20 \text{ seconds}$$

$$0 = \frac{80 \text{ seconds}}{n}$$

That is, there is *no amount* by which we can enhance multiply to achieve a fivefold increase in performance, if multiply accounts for only 80% of the workload. The performance enhancement possible with a given improvement is limited by the amount that the improved feature is used.

This concept also yields what we call the law of diminishing returns in everyday life. We can use Amdahl's law to estimate performance improvements when we know the time consumed for some function and its potential speed up. Amdahl's law, together with the CPU performance equation, are handy tools for evaluating potential enhancements. Amdahl's law is explored in more detail in the exercises and in the **In More Depth**: Amdahl's Law on the CD.

A common theme in hardware design is a corollary of Amdahl's law: *Make the common case fast*. This simple guideline reminds us that in many cases the frequency with which one event occurs may be much higher than another. Amdahl's law reminds us that the opportunity for improvement is affected by how much time the event consumes. Thus, making the common case fast will tend to enhance performance better than optimizing the rare case. Ironically, the common case is often simpler than the rare case and hence is often easier to enhance.

*Pitfall: Using a subset of the performance equation as a performance metric.*

We have already shown the fallacy of predicting performance based on simply one of clock rate, instruction count, or CPI. Another common mistake is to use two of the three factors to compare performance. Although using two of the three factors may be valid in a limited context, it is also easily misused. Indeed, nearly all proposed alternatives to the use of time as the performance metric have led eventually to misleading claims, distorted results, or incorrect interpretations.



**million instructions per second (MIPS)** A measurement of program execution speed based on the number of millions of instructions. MIPS is computed as the instruction count divided by the product of the execution time and  $10^6$ .

One alternative to time as the metric is **MIPS (million instructions per second)**. For a given program, MIPS is simply

$$\text{MIPS} = \frac{\text{Instruction count}}{\text{Execution time} \times 10^6}$$

This MIPS measurement is also called *native MIPS* to distinguish it from some alternative definitions of MIPS that we discuss in Section 4.8 on the CD.

Since MIPS is an instruction execution rate, MIPS specifies performance inversely to execution time; faster computers have a higher MIPS rating. The good news about MIPS is that it is easy to understand, and faster computers mean bigger MIPS, which matches intuition.

There are three problems with using MIPS as a measure for comparing computers. First, MIPS specifies the instruction execution rate but does not take into account the capabilities of the instructions. We cannot compare computers with different instruction sets using MIPS, since the instruction counts will certainly differ. In our earlier example examining the SPEC CFP2000 performance, using MIPS to compare the performance of the Pentium III and Pentium 4 would yield misleading results. Second, MIPS varies between programs on the same computer; thus a computer cannot have a single MIPS rating for all programs. Finally and most importantly, MIPS can vary inversely with performance! There are many examples of this anomalous behavior; one is given below.

### EXAMPLE

#### MIPS as a Performance Measure

Consider the computer with three instruction classes and CPI measurements from the last example on page 252. Now suppose we measure the code for the same program from two different compilers and obtain the following data:

| Code from  | Instruction counts (in billions)<br>for each instruction class |   |   |
|------------|----------------------------------------------------------------|---|---|
|            | A                                                              | B | C |
| Compiler 1 | 5                                                              | 1 | 1 |
| Compiler 2 | 10                                                             | 1 | 1 |

Assume that the computer's clock rate is 4 GHz. Which code sequence will execute faster according to MIPS? According to execution time?

First we find the execution time for the two different compilers using the following equation:

$$\text{Execution time} = \frac{\text{CPU clock cycles}}{\text{Clock rate}}$$

We can use an earlier formula for CPU clock cycles:

$$\text{CPU clock cycles} = \sum_{i=1}^n (\text{CPI}_i \times C_i)$$

$$\text{CPU clock cycles}_1 = (5 \times 1 + 1 \times 2 + 1 \times 3) \times 10^9 = 10 \times 10^9$$

$$\text{CPU clock cycles}_2 = (10 \times 1 + 1 \times 2 + 1 \times 3) \times 10^9 = 15 \times 10^9$$

Now, we find the execution time for the two compilers:

$$\text{Execution time}_1 = \frac{10 \times 10^9}{4 \times 10^9} = 2.5 \text{ seconds}$$

### ANSWER

## Chapter 4 Assessing and Understanding Performance

As examples such as this show, MIPS can fail to give a true picture of performance—even when comparing two versions of the same program on the same computer. In Section 2.7, we discuss other uses of the term *MIPS*, and how such usages can also be misleading.

### Check Yourself

Consider the following performance measurements for a program:

| Measurement       | Computer A | Computer B |
|-------------------|------------|------------|
| Instruction count | 10 billion | 8 billion  |
| Clock rate        | 4 GHz      | 4 GHz      |
| CPI               | 1.0        | 1.1        |

- Which computer has the higher MIPS rating?
- Which computer is faster?

## 4.6 Concluding Remarks

Although we have focused on performance and how to evaluate it in this chapter, designing only for performance without considering cost, functionality, and other requirements is unrealistic. All computer designers must balance performance and cost. Of course, there exists a domain of *high-performance design*, in which performance is the primary goal and cost is secondary. Much of the supercomputer and high-end server industry designs in this fashion. At the other extreme is *low-cost design*, typified by the embedded market, where cost and power take precedence over performance. In the middle are most desktop designs, as well as low-end servers; these computers require *cost/performance design*, in which the designer balances cost against performance. Examples from the desktop computer industry typify the kinds of trade-offs that designers in this region must live with.

We have seen in this chapter that there is a reliable method of determining and reporting performance, using the execution time of real programs as the metric. This execution time is related to other important measurements we can make by the following equation:

$$\frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$